文件：backend/src/ai_core/brain.py

```python
# -*- coding: utf-8 -*-
import asyncio
import json
import logging
import weakref
from datetime import datetime
from zoneinfo import ZoneInfo

import httpx
from backend.src.ai_core.llm_config import llm
from backend.src.ai_core.tools.knowledge import (
    search_knowledge_base, ingest_document,
    search_web_and_stage_knowledge,
    list_knowledge, update_knowledge, delete_knowledge,
)
from backend.src.ai_core.tools.portrait import read_portrait, update_portrait
from backend.src.ai_core.tools.skill import (
    read_skill, upsert_skill, list_skills, delete_skill, create_action_skill,
)
from backend.src.ai_core.tools.resource import generate_learning_resource
from backend.src.ai_core.tools.search import web_search
from backend.src.ai_core.tools.mcp_external import load_external_mcp_tools
from backend.src.ai_core.tools.image import generate_image
from backend.src.ai_core.tools.exam import generate_exam_questions
from backend.src.ai_core.tools.path import (
    list_learning_paths, get_learning_path_detail, enroll_learning_path,
    regenerate_learning_path, update_path_node, add_path_node, delete_path_node,
)
from backend.src.ai_core.tools.animation import generate_slide_animation
from backend.src.ai_core.tools.video_search import search_online_video
from backend.src.ai_core.tools.history import get_used_history
from backend.src.ai_core.tools.memory import search_memory
from backend.src.utils.prompt_loader import load_prompt
from pydantic import create_model, Field as PydanticField
try:
    from langchain_classic.agents import AgentExecutor, create_tool_calling_agent
except ModuleNotFoundError:
    from langchain.agents import AgentExecutor, create_tool_calling_agent
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.tools import StructuredTool
from langchain_core.messages import HumanMessage, AIMessage


def _inject_user_id(tool, user_id: str):
    """拷贝一个 tool，移除 user_id 参数并自动注入当前用户 ID"""
    original_coro = tool.coroutine
    if tool.args_schema:
        fields = {}
        for name, field_info in tool.args_schema.model_fields.items():
            if name != "user_id":
```

文件：backend/src/ai_core/brain.py

```
            fields[name] = (field_info.annotation, field_info)
        new_schema = create_model(f"{tool.name}_input", **fields) if fields else None
    else:
        new_schema = None

    desc = (tool.description or "").replace("user_id用户数字ID", "")
    desc = desc.replace("，，", "，").replace("，。", "。").replace("参数：，", "参数：").strip()

    async def _scoped(**kwargs):
        kwargs["user_id"] = user_id
        return await original_coro(**kwargs)

    _scoped.__name__ = tool.name
    return StructuredTool.from_function(
        coroutine=_scoped,
        name=tool.name,
        description=desc,
        args_schema=new_schema,
    )


def _inject_chat_group_id(tool, chat_group_id: int):
    """为 get_used_history 注入当前聊天组 ID"""
    original_coro = tool.coroutine
    if tool.args_schema:
        fields = {}
        for name, field_info in tool.args_schema.model_fields.items():
            if name not in ("chat_group_id",):
                fields[name] = (field_info.annotation, field_info)
        new_schema = create_model(f"{tool.name}_scoped_input", **fields) if fields else None
    else:
        new_schema = None

    async def _scoped(**kwargs):
        kwargs["chat_group_id"] = chat_group_id
        return await original_coro(**kwargs)

    _scoped.__name__ = tool.name
    return StructuredTool.from_function(
        coroutine=_scoped,
        name=tool.name,
        description=(tool.description or ""),
        args_schema=new_schema,
    )


_MAX_HISTORY_TURNS = 20

# —— 消息分类：按需加载工具行为指南 ——

```

文件：backend/src/ai_core/brain.py

```
_CREATE_TRIGGERS = [
    "生成学习", "生成资料", "生成文档", "做个PPT", "做PPT", "生成PPT",
    "整理成文档", "整理成PPT", "做成文档", "做成PPT",
    "生成思维导图", "生成脑图", "做思维导图", "做脑图",
    "帮我整理", "帮我总结", "帮我生成",
    "出题", "出几道", "出一些题", "练习题", "测验", "考试模拟",
    "做几道题", "来几道题", "做题", "习题", "试卷",
    "画一张", "画个", "生成图片", "生成一张图", "配图", "插图",
    "帮我画", "帮我生成图",
    "生成动画", "播放PPT", "演示PPT", "旁白", "念给我听",
    "搜视频", "找视频", "视频教程",
]

_MANAGE_TRIGGERS = [
    "学习路径", "课程路径", "学习计划", "选课", "有哪些路径",
    "加入路径", "路径管理", "修改节点", "添加节点", "删除节点",
    "重新规划路径", "路径不合适", "路径查看",
    "skill", "Skill", "自定义提示词", "修改提示词", "设置提示词",
    "恢复默认", "升级生成", "删除skill", "创建skill",
    "动作skill", "action skill", "添加能力", "添加工具",
]


def _classify_message(message: str) -> set[str]:
    """根据用户消息判断需要加载哪些工具行为指南"""
    cats = set()
    for t in _CREATE_TRIGGERS:
        if t in message:
            cats.add("create")
            break
    for t in _MANAGE_TRIGGERS:
        if t in message:
            cats.add("manage")
            break
    return cats


# —— 工具注册表：工具名 → 工厂函数(uid, gid) → 已注入的 LangChain Tool ——
TOOL_REGISTRY: dict[str, callable] = {
    "search_knowledge_base":     lambda uid, gid: _inject_user_id(search_knowledge_base, uid),
    "ingest_document":           lambda uid, gid: _inject_user_id(ingest_document, uid),
    "search_web_and_stage_knowledge": lambda uid, gid: _inject_user_id(search_web_and_stage_knowledge, uid),
    "list_knowledge":            lambda uid, gid: _inject_user_id(list_knowledge, uid),
    "update_knowledge":          lambda uid, gid: _inject_user_id(update_knowledge, uid),
    "delete_knowledge":          lambda uid, gid: _inject_user_id(delete_knowledge, uid),
    "read_portrait":             lambda uid, gid: _inject_user_id(read_portrait, uid),
    "update_portrait":           lambda uid, gid: _inject_user_id(update_portrait, uid),
    "get_used_history":          lambda uid, gid: _inject_chat_group_id(_inject_user_id(get_used_history, uid), gid…
    "search_memory":             lambda uid, gid: _inject_user_id(search_memory, uid),
    "web_search":                lambda uid, gid: web_search,
```

文件：backend/src/ai_core/brain.py

```
    "read_skill":              lambda uid, gid: _inject_user_id(read_skill, uid),
    "upsert_skill":            lambda uid, gid: _inject_user_id(upsert_skill, uid),
    "list_skills":             lambda uid, gid: _inject_user_id(list_skills, uid),
    "delete_skill":            lambda uid, gid: _inject_user_id(delete_skill, uid),
    "create_action_skill":     lambda uid, gid: _inject_user_id(create_action_skill, uid),
    "generate_learning_resource": lambda uid, gid: _inject_chat_group_id(_inject_user_id(generate_learning_resource,…
    "generate_image":          lambda uid, gid: _inject_chat_group_id(_inject_user_id(generate_image, uid), gid),
    "generate_exam_questions":  lambda uid, gid: _inject_chat_group_id(_inject_user_id(generate_exam_questions, ui…
    "generate_slide_animation": lambda uid, gid: _inject_chat_group_id(_inject_user_id(generate_slide_animation, u…
    "search_online_video":     lambda uid, gid: _inject_chat_group_id(_inject_user_id(search_online_video, uid), …
    "list_learning_paths":     lambda uid, gid: _inject_user_id(list_learning_paths, uid),
    "get_learning_path_detail": lambda uid, gid: _inject_user_id(get_learning_path_detail, uid),
    "enroll_learning_path":    lambda uid, gid: _inject_user_id(enroll_learning_path, uid),
    "regenerate_learning_path": lambda uid, gid: _inject_user_id(regenerate_learning_path, uid),
    "update_path_node":        lambda uid, gid: _inject_user_id(update_path_node, uid),
    "add_path_node":           lambda uid, gid: _inject_user_id(add_path_node, uid),
    "delete_path_node":        lambda uid, gid: _inject_user_id(delete_path_node, uid),
}


class Brain:
    _instances: weakref.WeakSet = weakref.WeakSet()

    def __init__(self, user_id: int, chat_group_id: int | None = None,
                 session_id: str | None = None, agent_id: int | None = None):
        self.user_id = user_id
        self.chat_group_id = chat_group_id
        self.session_id = session_id or f"brain_{user_id}"
        self.agent_id = agent_id
        self._agent_persona: str | None = None
        self._agent_tool_names: set[str] | None = None
        self._agent_memory_text: str = ""
        self._raw_executor = None
        self._action_tools_loaded = False
        self._agent_config_loaded = False
        self._history: list = []
        self._history_hydrated = False  # 是否已从 DB 水合最近 N 轮（幂等）
        Brain._instances.add(self)

    # —— 记忆水合 ——

    async def hydrate_history(self, before_id: int | None = None):
        """冷启动时从 DB 水合最近 N 轮历史（幂等）。

        配合多级记忆系统：跨会话恢复短期记忆。before_id 用于截止（如当前
        消息 id），避免把正在处理的这一轮当成历史重放。
        """
        if self._history_hydrated:
            return
        self._history_hydrated = True
```

文件：backend/src/ai_core/brain.py

```
        try:
            from backend.src.models.chat_history_model import ChatHistory
            qs = ChatHistory.filter(
                user_id=self.user_id, chat_group_id=self.chat_group_id or 0
            )
            if before_id:
                qs = qs.filter(id__lt=before_id)
            records = await qs.order_by("-id").limit(_MAX_HISTORY_TURNS).all()
            for r in reversed(records):
                if r.req:
                    self._history.append(HumanMessage(content=r.req))
                if r.res:
                    self._history.append(AIMessage(content=r.res))
        except Exception:
            logging.getLogger(__name__).exception("水合历史失败 user_id=%s", self.user_id)

    # ── 动态工具工厂 ──

    @staticmethod
    def _make_http_tool(skill: dict):
        """将 HTTP 类型的 action skill 包装成 LangChain StructuredTool"""
        config = json.loads(skill["action_config"]) if isinstance(skill["action_config"], str) else skill["action_con…
        safe_name = skill["name"].replace("-", "_").replace(" ", "_")

        async def _handler(**kwargs):
            url = config["url"]
            for k, v in kwargs.items():
                url = url.replace(f"{{{{{k}}}}}", str(v))
            timeout = httpx.Timeout(30.0)
            async with httpx.AsyncClient(timeout=timeout) as client:
                method = config.get("method", "GET").upper()
                resp = await client.request(method, url)
                text = resp.text[:3000]
                if resp.status_code >= 400:
                    return f"请求失败 (HTTP {resp.status_code}): {text}"
                return text

        _handler.__name__ = safe_name

        params_schema = config.get("params", {})
        args_schema = None
        if params_schema:
            fields = {}
            for pname, pdesc in params_schema.items():
                if isinstance(pdesc, dict):
                    desc = str(pdesc.get("description") or pdesc.get("desc") or "")
                else:
                    desc = str(pdesc or "")
                fields[pname] = (str, PydanticField(description=desc))
            args_schema = create_model(f"{safe_name}_input", **fields)
```

文件：backend/src/ai_core/brain.py

```
251
252         return StructuredTool.from_function(
253             coroutine=_handler,
254             name=safe_name,
255             description=skill.get("tool_description", "") or f"自定义技能: {skill['name']}",
256             args_schema=args_schema,
257         )
258
259     async def _load_action_tools_async(self):
260         """在正确的 async 上下文中从 DB 加载 action skill"""
261         from backend.src.service.skill import service as skill_service
262         skills = await skill_service.list_actions(user_id=self.user_id)
263         tools = []
264         for s in skills:
265             if s.get("action_type") != "http":
266                 continue
267             try:
268                 tools.append(self._make_http_tool(s))
269             except Exception:
270                 logging.getLogger(__name__).exception("action skill 构造失败，已跳过: %s", s.get("name"))
271         return tools
272
273     # —— 热刷新 ——
274
275     @classmethod
276     def rebuild_for_user(cls, user_id: int):
277         """创建/删除 action skill 后标记需要刷新，下次对话时自动重建"""
278         for inst in cls._instances:
279             if inst.user_id == user_id:
280                 inst._action_tools_loaded = False
281                 inst._agent_config_loaded = False
282
283     async def _load_agent_config(self):
284         """Load user-defined agent config: persona, tool whitelist, memory.
285         Only runs once when agent_id is set and not yet loaded."""
286         if self.agent_id is None or self._agent_config_loaded:
287             return
288         try:
289             from backend.src.service.agent.service import get as get_agent, get_memory_text
290             agent_config = await get_agent(self.user_id, self.agent_id)
291             if agent_config:
292                 self._agent_persona = agent_config.get("persona", "") or None
293                 tool_names = agent_config.get("tools", [])
294                 self._agent_tool_names = set(tool_names) if tool_names else None
295                 self._agent_memory_text = await get_memory_text(self.user_id, self.agent_id)
296         except Exception:
297             logging.getLogger(__name__).exception("加载智能体配置失败 agent_id=%s", self.agent_id)
298             self._agent_persona = None
299             self._agent_tool_names = None
300             self._agent_memory_text = ""
```

文件：backend/src/ai_core/brain.py

```
        self._agent_config_loaded = True

    def _build_agent(self, action_tools: list, mcp_tools: list):
        now = datetime.now(ZoneInfo("Asia/Shanghai"))
        tz_name = "Asia/Shanghai"
        date_str = now.strftime("%Y-%m-%d")
        time_str = now.strftime("%Y-%m-%d %H:%M:%S")
        current_time_context = (
            f"\n\n### Current Time Anchor\n"
            f"- Date: {date_str}\n"
            f"- Time: {time_str}\n"
            f"- Timezone: {tz_name}\n"
            f"- Always use this date as reference for time-sensitive queries.\n"
        )

        if self._agent_persona:
            system_prompt = (
                self._agent_persona
                + current_time_context
                + (("\n" + self._agent_memory_text) if self._agent_memory_text else "")
                + "\n\n{path_context}\n\n{portrait_context}\n\n{memory_context}"
                + "\n\n## Output Rules\n"
                + "- Use Markdown for formatting, NOT raw HTML tags.\n"
                + "- Wrap inline math in $...$ and display math in $$...$$.\n"
                + "- Never output <br>, <div>, <span> or other HTML tags.\n"
            )
        else:
            system_prompt = load_prompt("chat/unified") + current_time_context

        prompt = ChatPromptTemplate.from_messages([
            ("system", system_prompt),
            MessagesPlaceholder(variable_name="history"),
            ("human", "{input}"),
            MessagesPlaceholder(variable_name="agent_scratchpad"),
        ])

        uid = str(self.user_id)
        gid = self.chat_group_id or 0

        if self._agent_tool_names is not None:
            tool_names_to_load = self._agent_tool_names
        else:
            tool_names_to_load = set(TOOL_REGISTRY.keys())

        tools = []
        for name in tool_names_to_load:
            factory = TOOL_REGISTRY.get(name)
            if factory:
                tools.append(factory(uid, gid))

```

文件：backend/src/ai_core/brain.py

```
        tools.extend(_inject_user_id(t, uid) for t in action_tools)
        tools.extend(mcp_tools)

        agent = create_tool_calling_agent(llm=llm, prompt=prompt, tools=tools)
        max_iters = max(8, len(tools) * 2)
        self._raw_executor = AgentExecutor(
            agent=agent, tools=tools,
            verbose=True, handle_parsing_errors=True, max_iterations=max_iters,
        )

    def _load_tool_guides(self, message: str) -> str:
        """根据消息内容按需加载工具行为指南；未命中则返回空字符串"""
        cats = _classify_message(message)
        parts: list[str] = []
        if "create" in cats:
            parts.append(load_prompt("chat/guide_create"))
        if "manage" in cats:
            parts.append(load_prompt("chat/guide_manage"))
        return "\n".join(parts)

    async def _ensure_action_tools(self):
        """首次调用或 rebuild_for_user 后，异步加载 agent 配置、action tools 并重建 agent"""
        if self._action_tools_loaded:
            return
        await self._load_agent_config()
        try:
            action_tools = await self._load_action_tools_async()
        except Exception:
            logging.getLogger(__name__).exception("加载 action tools 失败")
            action_tools = []
        try:
            mcp_tools = await load_external_mcp_tools()
        except Exception:
            logging.getLogger(__name__).exception('Failed to load MCP tools')
            mcp_tools = []
        self._build_agent(action_tools, mcp_tools)
        self._action_tools_loaded = True

    async def chat(self, message: str, resource_context: str = "", path_context: str = "", portrait_context: str = ""…
        await self._ensure_action_tools()
        tool_guides = self._load_tool_guides(message)
        response = await self._raw_executor.ainvoke({
            "input": message,
            "history": list(self._history),
            "current_user_id": str(self.user_id),
            "resource_context": resource_context,
            "path_context": path_context,
            "portrait_context": portrait_context,
            "memory_context": memory_context,
            "tool_guides": tool_guides,
```

文件：backend/src/ai_core/brain.py

```
        })
        self._history.append(HumanMessage(content=message))
        self._history.append(AIMessage(content=response["output"]))
        if len(self._history) > _MAX_HISTORY_TURNS * 2:
            self._history = self._history[-_MAX_HISTORY_TURNS * 2:]
        return response["output"]

    async def stream(self, message: str, resource_context: str = "", path_context: str = "", portrait_context: str = …
        """逐 token 流式输出 — 包含工具调用事件，工具执行期间自动心跳保活"""
        await self._ensure_action_tools()

        full_response = ""
        tool_running = False
        tool_guides = self._load_tool_guides(message)

        async def _stream_events(version: str):
            nonlocal tool_running
            agen = self._raw_executor.astream_events(
                {
                    "input": message,
                    "history": list(self._history),
                    "current_user_id": str(self.user_id),
                    "resource_context": resource_context,
                    "path_context": path_context,
                    "portrait_context": portrait_context,
                    "memory_context": memory_context,
                    "tool_guides": tool_guides,
                },
                version=version,
            )
            while True:
                try:
                    event = await asyncio.wait_for(agen.__anext__(), timeout=30 if tool_running else 120)
                except asyncio.TimeoutError:
                    yield {"type": "keepalive"}
                    continue
                except StopAsyncIteration:
                    break
                yield event

        try:
            async for event in _stream_events("v2"):
                kind = event.get("event", "")

                if kind == "on_tool_start":
                    tool_running = True
                    tool_name = event.get("name", "")
                    yield {"role": "tool", "type": "tool_start", "tool": tool_name}

                elif kind == "on_tool_end":
```

文件：backend/src/ai_core/brain.py

```
                tool_running = False
                tool_name = event.get("name", "")
                tool_output = event.get("data", {}).get("output", "")
                if isinstance(tool_output, str) and len(tool_output) > 500:
                    tool_output = tool_output[:500] + "..."
                yield {"role": "tool", "type": "tool_end", "tool": tool_name, "output": str(tool_output)}

            elif kind == "on_chat_model_stream":
                chunk = event.get("data", {}).get("chunk")
                if chunk:
                    content = getattr(chunk, "content", None)
                    if content:
                        full_response += content
                        yield {"role": "assistant", "type": "chunk", "content": content}
        except (TypeError, NotImplementedError):
            async for event in _stream_events("v1"):
                kind = event.get("event", "")

                if kind == "on_tool_start":
                    tool_running = True
                    tool_name = event.get("name", "")
                    yield {"role": "tool", "type": "tool_start", "tool": tool_name}

                elif kind == "on_tool_end":
                    tool_running = False
                    tool_name = event.get("name", "")
                    tool_output = event.get("data", {}).get("output", "")
                    if isinstance(tool_output, str) and len(tool_output) > 500:
                        tool_output = tool_output[:500] + "..."
                    yield {"role": "tool", "type": "tool_end", "tool": tool_name, "output": str(tool_output)}

                elif kind == "on_chat_model_stream":
                    chunk = event.get("data", {}).get("chunk")
                    if chunk:
                        content = getattr(chunk, "content", None)
                        if content:
                            full_response += content
                            yield {"role": "assistant", "type": "chunk", "content": content}

        self._history.append(HumanMessage(content=message))
        self._history.append(AIMessage(content=full_response))
        if len(self._history) > _MAX_HISTORY_TURNS * 2:
            self._history = self._history[-_MAX_HISTORY_TURNS * 2:]
```

文件：backend/src/ai_core/classroom_graph.py

```
"""
LangGraph 双智能体编排 — 互动课堂脚本生成
WriterAgent(导演规划 → 并行分幕) → ReviewerAgent(独立审核 → 带反馈重写)
"""
import asyncio
import json
import logging
```

文件：backend/src/ai_core/classroom_graph.py

```
import re
import time
from typing import Any, TypedDict, NotRequired

from langgraph.graph import StateGraph, START, END

from backend.src.ai_core.llm_config import llm
from backend.src.utils.prompt_loader import load_prompt, fill_prompt
from backend.src.utils.json_parser import parse_llm_json
from backend.src.service.path.classroom import (
    _classroom_segment_narration_text,
    _normalize_lesson,
    _GENERIC_CLASSROOM_PHRASES,
    _content_evidence_terms,
    generate_classroom_audio,
)

logger = logging.getLogger(__name__)

# ═══════════════════════════════════════════════════════════════
#  常量
# ═══════════════════════════════════════════════════════════════

# 固定四幕，顺序不可变（与 _fallback_lesson / 前端契约一致）
_SEGMENT_IDS = ("lead-in", "concept", "exercise", "feynman")

# 幕 id → segment.type（前端 isResourceScene/isQuizScene/isFeynmanScene 依赖）
_SEGMENT_TYPES = {
    "lead-in": "hook",
    "concept": "concept",
    "exercise": "exercise",
    "feynman": "feynman",
}

# 审核不通过时只重写被点名的模块，最多进行一轮定向优化。
_MAX_REVIEW_RETRIES = 1
_REVIEW_PASS_SCORE = 80      # 通过阈值：score >= 80 且结构契约通过
_MAX_SEGMENT_CONCURRENCY = 4  # 四幕并行生成
_REVIEW_FEEDBACK_MAX_LEN = 400 # 回灌给 writer 的总体意见截断，防止 prompt 膨胀
_TTS_PREWARM_CONCURRENCY = 2
_tts_prewarm_sem = asyncio.Semaphore(_TTS_PREWARM_CONCURRENCY)
_tts_prewarm_tasks: set[asyncio.Task] = set()

# 蓝图缺失时的幕默认 goal / focus（保证任何情况都产出"四幕齐全"蓝图）
_DEFAULT_GOALS = {
    "lead-in": "结合学生画像说明这个知识点与学生的关系",
    "concept": "讲清知识点的定义、结构、关系和使用边界",
    "exercise": "用一道真实练习检查概念、步骤或易错点",
    "feynman": "用自己的话反讲，并在右侧对话中补齐漏洞",
}
```

文件：backend/src/ai_core/classroom_graph.py

```
58  _DEFAULT_FOCUS = {
59      "lead-in": "用户专业、年级、目标与该知识点的具体联系",
60      "concept": "节点中的核心术语、公式、步骤、例子和易错点",
61      "exercise": "节点测验中的题干、选项、答案和解析",
62      "feynman": "三句话反讲任务、一个例子、一次右侧对话追问",
63  }
64
65  _SCENE_RESPONSIBILITIES = {
66      "lead-in": "结合学生画像或学习目标，说明该知识点与学生的具体关系。",
67      "concept": "讲清知识点的定义、结构、关系、公式、步骤或边界。",
68      "exercise": "说明现有练习将检查的知识点、判断依据或易错点。",
69      "feynman": "给出具体反讲任务，要求学生在右侧对话框作答。",
70  }
71
72
73  # ═══════════════════════════════════════════════════════════════
74  #  State
75  # ═══════════════════════════════════════════════════════════════
76
77  class ClassroomState(TypedDict):
78      # ── 输入（generate_classroom_lesson 注入）──
79      path_id: int
80      node_id: int
81      user_id: int
82      subject: str
83      topic: str
84      summary: str
85      knowledge_tags: list[str]
86      quiz_config: dict[str, Any]
87      quiz_snapshot: dict[str, Any]
88      resources: list[dict[str, str]]
89      portrait_context: str
90      fallback_lesson: dict[str, Any]   # _fallback_lesson(...) 预计算结果，异常兜底用
91      llm_priority: NotRequired[str]    # 默认 "high"
92      max_retries: NotRequired[int]     # 默认 _MAX_REVIEW_RETRIES
93      trace_id: NotRequired[str]        # 一次课堂请求的日志关联标识
94
95      # ── Writer 输出 ──
96      teaching_outline: NotRequired[dict[str, Any]] # 导演产物；重写轮次复用，不重复规划
97      raw_lesson: NotRequired[dict[str, Any]]       # 4 幕原始拼接（归一化前），Reviewer 审核对象
98      lesson: NotRequired[dict[str, Any]]           # _normalize_lesson 归一化后的最终课堂
99
100     # ── Reviewer 输出 ──
101     review_passed: NotRequired[bool]
102     review_score: NotRequired[float]
103     review_feedback: NotRequired[str]           # 未通过时的总体意见
104     review_issues: NotRequired[list[dict[str, Any]]] # [{segment_id, category, message}]
105     retry_count: NotRequired[int]
106
107
```

文件：backend/src/ai_core/classroom_graph.py

```
# ═══════════════════════════════════════════════════════════
# Writer — 导演规划 + 并行分幕
# ═══════════════════════════════════════════════════════════

async def writer_node(state: ClassroomState) -> dict:
    """首次先规划教学主线，再并行生成四幕；重写轮次复用蓝图并带审核意见重写。"""
    started_at = time.perf_counter()
    path_id = state.get("path_id")
    node_id = state.get("node_id")
    retry_count = state.get("retry_count", 0)
    trace_id = state.get("trace_id", "-")
    logger.info("[ClassroomWriter] 开始生成 trace=%s path=%s node=%s retry=%s", trace_id, path_id, node_id, retry_count)
    try:
        outline = state.get("teaching_outline")
        if outline is None:
            outline = await _plan_teaching_outline(state)
        raw_lesson = await _generate_segments(state, outline)
        lesson = _normalize_lesson(
            raw_lesson,
            state.get("fallback_lesson"),
            topic=state.get("topic", ""),
            summary=state.get("summary", ""),
            knowledge_tags=state.get("knowledge_tags", []),
            resources=state.get("resources", []),
        )
        logger.info(
            "[ClassroomWriter] 生成完成 trace=%s path=%s node=%s retry=%s segments=%s elapsed=%.2fs",
            trace_id,
            path_id,
            node_id,
            retry_count,
            len(raw_lesson.get("segments", [])),
            time.perf_counter() - started_at,
        )
        return {"teaching_outline": outline, "raw_lesson": raw_lesson, "lesson": lesson}
    except Exception:
        logger.exception("[ClassroomWriter] 生成失败 trace=%s path=%s node=%s elapsed=%.2fs", trace_id, path_id, node_id,…
        return {"lesson": {}, "raw_lesson": {}, "review_passed": False}


async def _plan_teaching_outline(state: ClassroomState) -> dict:
    """导演阶段：1 次 LLM 调用产出四幕蓝图和本节总结。解析失败时用默认蓝图。"""
    started_at = time.perf_counter()
    user_id_int = int(state.get("user_id", 0))
    llm_priority = state.get("llm_priority", "high")
    prompt_text = fill_prompt(
        load_prompt("classroom/writer_planner"),
        subject=state.get("subject", "未知"),
        topic=state.get("topic", ""),
        summary=state.get("summary", ""),
```

文件：backend/src/ai_core/classroom_graph.py

```
        knowledge_tags=json.dumps(state.get("knowledge_tags", []), ensure_ascii=False),
        quiz_config=json.dumps(state.get("quiz_config", {}), ensure_ascii=False),
        quiz_snapshot=json.dumps(state.get("quiz_snapshot", {}), ensure_ascii=False)[:900],
        resources_json=json.dumps(state.get("resources", []), ensure_ascii=False)[:2200],
        evidence_terms=json.dumps(
            _content_evidence_terms(
                state.get("topic", ""),
                state.get("summary", ""),
                *(state.get("knowledge_tags", []) or []),
                *[
                    value
                    for resource in (state.get("resources", []) or [])
                    if isinstance(resource, dict)
                    for value in (resource.get("title"), resource.get("summary"))
                ],
            ),
            ensure_ascii=False,
        ),
        portrait_context=state.get("portrait_context", "暂无画像数据"),
    )
    raw: dict = {}
    try:
        logger.info("[ClassroomWriter] 蓝图请求模型 trace=%s path=%s node=%s", state.get("trace_id", "-"), state.get("path_…
        response = await llm.ainvoke(prompt_text, priority=llm_priority, user_id=user_id_int, pool="classroom")
        parsed = parse_llm_json(response.content)
        if isinstance(parsed, dict):
            raw = parsed
    except Exception:
        logger.exception("[ClassroomWriter] 蓝图规划失败 trace=%s path=%s node=%s", state.get("trace_id", "-"), state.get("…
    outline = _normalize_outline(raw, state)
    logger.info(
        "[ClassroomWriter] 蓝图完成 trace=%s path=%s node=%s source=%s elapsed=%.2fs",
        state.get("trace_id", "-"),
        state.get("path_id"),
        state.get("node_id"),
        "llm" if raw else "default",
        time.perf_counter() - started_at,
    )
    return outline


def _normalize_outline(raw: dict, state: ClassroomState) -> dict:
    """按 _SEGMENT_IDS 固定顺序重排/去重 segments，缺失的幕用默认 goal/focus/style 补齐。"""
    topic = state.get("topic", "当前知识点")
    default_segments = [
        {
            "id": sid,
            "goal": _DEFAULT_GOALS.get(sid, "完成本幕讲解"),
            "focus": _DEFAULT_FOCUS.get(sid, f"围绕 {topic} 展开"),
            "style": "清晰、分步、贴合画像",
```

文件：backend/src/ai_core/classroom_graph.py

```
        }
        for sid in _SEGMENT_IDS
    ]
    by_id: dict[str, dict] = {}
    segs = raw.get("segments") if isinstance(raw.get("segments"), list) else []
    for item in segs:
        if not isinstance(item, dict):
            continue
        sid = item.get("id")
        if sid not in _SEGMENT_TYPES:
            continue
        by_id[str(sid)] = {
            "goal": str(item.get("goal") or _DEFAULT_GOALS.get(sid, "")),
            "focus": str(item.get("focus") or _DEFAULT_FOCUS.get(sid, f"围绕 {topic} 展开")),
            "style": str(item.get("style") or "清晰、分步、贴合画像"),
        }
    merged = []
    for default in default_segments:
        merged.append(by_id.get(default["id"], default))
    takeaways = raw.get("key_takeaways") if isinstance(raw.get("key_takeaways"), list) else []
    return {
        "title": str(raw.get("title") or topic),
        "personal_summary": str(raw.get("personal_summary") or f"围绕 {topic} 完成一次互动课堂。"),
        "main_thread": str(raw.get("main_thread") or f"先分清{topic}要解决的问题，再拆开核心关系，用资料验证，最后反讲。"),
        "learning_summary": str(raw.get("learning_summary") or f"本节围绕“{topic}”理解核心概念，并用资料核对后完成一次反讲。"),
        "key_takeaways": [str(item) for item in takeaways if str(item).strip()][:4],
        "feynman_plan": raw.get("feynman_plan") if isinstance(raw.get("feynman_plan"), dict) else {},
        "segments": merged,
    }


async def _generate_segments(state: ClassroomState, outline: dict) -> dict:
    """首轮生成四幕；审核后的定向优化只重写被点名的幕。"""
    started_at = time.perf_counter()
    sem = asyncio.Semaphore(_MAX_SEGMENT_CONCURRENCY)
    issues = state.get("review_issues") or []
    global_feedback = state.get("review_feedback") or ""
    retry_count = state.get("retry_count", 0)
    previous_raw = state.get("raw_lesson") if isinstance(state.get("raw_lesson"), dict) else {}
    previous_by_id = {
        str(segment.get("id")): segment
        for segment in previous_raw.get("segments", [])
        if isinstance(segment, dict) and str(segment.get("id")) in _SEGMENT_IDS
    }
    targeted_ids = {
        str(issue.get("segment_id"))
        for issue in issues
        if isinstance(issue, dict) and str(issue.get("segment_id")) in _SEGMENT_IDS
    }
    is_targeted_retry = retry_count > 0 and len(previous_by_id) == len(_SEGMENT_IDS) and bool(targeted_ids)
```

文件：backend/src/ai_core/classroom_graph.py

```
258
259     if retry_count > 0 and not is_targeted_retry:
260         logger.warning(
261             "[ClassroomWriter] 审核未给出可定位模块，不重写整课 path=%s node=%s retry=%s",
262             state.get("path_id"), state.get("node_id"), retry_count,
263         )
264         return previous_raw
265
266     parallel_ids = _SEGMENT_IDS[:-1]
267     logger.info(
268         "[ClassroomWriter] 并行幕开始 trace=%s path=%s node=%s scenes=%s mode=%s",
269         state.get("trace_id", "-"),
270         state.get("path_id"), state.get("node_id"), ",".join(parallel_ids),
271         "targeted" if is_targeted_retry else "initial",
272     )
273     parallel_jobs = [
274         _gen_segment(state, outline, sid, index, sem, issues, global_feedback)
275         for index, sid in enumerate(parallel_ids)
276         if not is_targeted_retry or sid in targeted_ids
277     ]
278     generated_parallel = await asyncio.gather(*parallel_jobs) if parallel_jobs else []
279     generated_by_id = {str(segment.get("id")): segment for segment in generated_parallel if isinstance(segment, dict)}
280     parallel_segments = [generated_by_id.get(sid) or previous_by_id.get(sid, {}) for sid in parallel_ids]
281     logger.info(
282         "[ClassroomWriter] 并行幕完成 trace=%s path=%s node=%s valid=%s/%s elapsed=%.2fs",
283         state.get("trace_id", "-"),
284         state.get("path_id"),
285         state.get("node_id"),
286         sum(bool(segment) for segment in parallel_segments),
287         len(parallel_ids),
288         time.perf_counter() - started_at,
289     )
290     feynman_started_at = time.perf_counter()
291     should_rewrite_feynman = not is_targeted_retry or _SEGMENT_IDS[-1] in targeted_ids
292     if should_rewrite_feynman:
293         logger.info("[ClassroomWriter] 费曼幕开始 trace=%s path=%s node=%s", state.get("trace_id", "-"), state.get("path_i…
294         feynman = await _gen_segment(
295             state,
296             outline,
297             _SEGMENT_IDS[-1],
298             len(_SEGMENT_IDS) - 1,
299             sem,
300             issues,
301             global_feedback,
302             completed_segments=parallel_segments,
303         )
304     else:
305         feynman = previous_by_id.get(_SEGMENT_IDS[-1], {})
306         logger.info("[ClassroomWriter] 费曼幕复用 path=%s node=%s", state.get("path_id"), state.get("node_id"))
307     logger.info(
```

文件：backend/src/ai_core/classroom_graph.py

```
        "[ClassroomWriter] 费曼幕完成 trace=%s path=%s node=%s valid=%s elapsed=%.2fs total=%.2fs",
        state.get("trace_id", "-"),
        state.get("path_id"),
        state.get("node_id"),
        bool(feynman),
        time.perf_counter() - feynman_started_at,
        time.perf_counter() - started_at,
    )
    raw_segments = parallel_segments + [feynman]
    return {
        "title": outline.get("title", state.get("topic", "")),
        "personal_summary": outline.get("personal_summary", ""),
        "learning_summary": outline.get("learning_summary", ""),
        "key_takeaways": outline.get("key_takeaways", []),
        "segments": raw_segments,
    }


async def _gen_segment(
    state: ClassroomState,
    outline: dict,
    scene_id: str,
    index: int,
    sem: asyncio.Semaphore,
    issues: list[dict],
    global_feedback: str,
    completed_segments: list[dict] | None = None,
) -> dict:
    """生成单幕；费曼幕可接收前三幕的压缩结果。"""
    started_at = time.perf_counter()
    segs = outline.get("segments") or []
    info = segs[index] if index < len(segs) else {}
    feynman_plan = json.dumps(outline.get("feynman_plan") or {}, ensure_ascii=False)
    prompt_text = fill_prompt(
        load_prompt("classroom/writer_segment"),
        subject=state.get("subject", "未知"),
        topic=state.get("topic", ""),
        summary=state.get("summary", ""),
        knowledge_tags=json.dumps(state.get("knowledge_tags", []), ensure_ascii=False),
        portrait_context=state.get("portrait_context", "暂无画像数据"),
        resources_json=json.dumps(state.get("resources", []), ensure_ascii=False)[:1800],
        evidence_terms=json.dumps(
            _content_evidence_terms(
                state.get("topic", ""),
                state.get("summary", ""),
                *(state.get("knowledge_tags", []) or []),
                *[
                    value
                    for resource in (state.get("resources", []) or [])
                    if isinstance(resource, dict)
```

文件：backend/src/ai_core/classroom_graph.py

```
                for value in (resource.get("title"), resource.get("summary"))
              ],
            ),
            ensure_ascii=False,
          ),
          scene_id=scene_id,
          scene_index=index + 1,
          scene_total=len(_SEGMENT_IDS),
          scene_goal=info.get("goal", ""),
          scene_focus=info.get("focus", ""),
          scene_style=info.get("style", ""),
          previous_scene_focus=_seg_focus(outline, index - 1),
          next_scene_focus=_seg_focus(outline, index + 1),
          main_thread=outline.get("main_thread", ""),
          feynman_plan=feynman_plan,
          completed_segments=_completed_segment_context(completed_segments),
          review_feedback=_feedback_for_segment(scene_id, issues, global_feedback),
          forbidden_phrases=json.dumps(_GENERIC_CLASSROOM_PHRASES, ensure_ascii=False),
        )
        user_id_int = int(state.get("user_id", 0))
        llm_priority = state.get("llm_priority", "high")
        queue_started_at = time.perf_counter()
        try:
          logger.info("[ClassroomWriter] 单幕等待并发槽 trace=%s path=%s node=%s scene=%s", state.get("trace_id", "-"), state.…
          async with sem:
            logger.info("[ClassroomWriter] 单幕请求模型 trace=%s path=%s node=%s scene=%s queue_wait=%.2fs", state.get("tra…
            response = await llm.ainvoke(prompt_text, priority=llm_priority, user_id=user_id_int, pool="classroom")
          parsed = parse_llm_json(response.content)
          if not isinstance(parsed, dict):
            logger.warning("[ClassroomWriter] 单幕返回非 JSON trace=%s path=%s node=%s scene=%s elapsed=%.2fs", state.get(…
            return {}
          parsed["id"] = scene_id
          parsed["type"] = _SEGMENT_TYPES.get(scene_id, scene_id)
          narration_text = _classroom_segment_narration_text(parsed)
          if narration_text:
            task = asyncio.create_task(
              _prewarm_segment_audio(
                user_id=user_id_int,
                path_id=int(state.get("path_id", 0)),
                node_id=int(state.get("node_id", 0)),
                scene_id=scene_id,
                text=narration_text,
              )
            )
            _tts_prewarm_tasks.add(task)
            task.add_done_callback(_tts_prewarm_tasks.discard)
          logger.info("[ClassroomWriter] 单幕完成 trace=%s path=%s node=%s scene=%s chars=%s elapsed=%.2fs", state.get("tra…
          return parsed
        except Exception:
          logger.exception("[ClassroomWriter] 单幕生成失败 trace=%s path=%s node=%s scene=%s elapsed=%.2fs", state.get("trace…
```

文件：backend/src/ai_core/classroom_graph.py

```
408         return {}
409
410
411 async def _prewarm_segment_audio(
412     user_id: int,
413     path_id: int,
414     node_id: int,
415     scene_id: str,
416     text: str,
417 ) -> None:
418     """分幕生成后立即预热语音；失败不影响课堂正文和审核。"""
419     try:
420         async with _tts_prewarm_sem:
421             await generate_classroom_audio(text, user_id)
422         logger.info(
423             "[ClassroomTTS] 预热完成 path=%s node=%s scene=%s chars=%s",
424             path_id,
425             node_id,
426             scene_id,
427             len(text),
428         )
429     except Exception:
430         logger.exception(
431             "[ClassroomTTS] 预热失败 path=%s node=%s scene=%s",
432             path_id,
433             node_id,
434             scene_id,
435         )
436
437
438 def _completed_segment_context(segments: list[dict] | None) -> str:
439     """只把前三幕的关键内容交给费曼幕，避免把完整 JSON 塞进 prompt。"""
440     if not segments:
441         return "前三幕尚未完成；当前不是费曼幕时忽略此字段。"
442     compact = []
443     for segment in segments:
444         if not isinstance(segment, dict):
445             continue
446         compact.append({
447             "id": segment.get("id", ""),
448             "title": segment.get("title", ""),
449             "teacher_speech": segment.get("teacher_speech") or segment.get("script", ""),
450             "points": (segment.get("points") or segment.get("board_items") or [])[:4],
451             "example": segment.get("example", ""),
452             "question": (segment.get("question") or {}).get("prompt", ""),
453         })
454     return json.dumps(compact, ensure_ascii=False)[:5000] if compact else "前三幕没有返回可用内容。"
455
456
457 def _seg_focus(outline: dict, index: int) -> str:
```

文件：backend/src/ai_core/classroom_graph.py

```
    segs = outline.get("segments") or []
    if 0 <= index < len(segs):
        return str(segs[index].get("focus") or "")
    return ""


def _feedback_for_segment(segment_id: str, issues: list[dict], global_feedback: str) -> str:
    """把 issues 按 segment_id 过滤（None 视为全局）拼成该幕专属反馈，总体意见附加为末尾。"""
    parts: list[str] = []
    for issue in issues:
        if not isinstance(issue, dict):
            continue
        sid = issue.get("segment_id")
        if sid is None or str(sid) == segment_id:
            msg = str(issue.get("message") or "")
            if msg:
                parts.append(msg)
    if global_feedback:
        parts.append(f"[总体意见] {global_feedback}")
    return "\n".join(parts) if parts else ""


# ═══════════════════════════════════════════════════════════════
#  Reviewer — 独立审核（对象是归一化前的 raw_lesson）
# ═══════════════════════════════════════════════════════════════

async def reviewer_node(state: ClassroomState) -> dict:
    """四个审核智能体并行检查分幕；只把问题回灌给对应模块。"""
    started_at = time.perf_counter()
    trace_id = state.get("trace_id", "-")
    raw_lesson = state.get("raw_lesson") or state.get("lesson")
    if not raw_lesson or not isinstance(raw_lesson.get("segments"), list):
        logger.warning("[ClassroomReviewer] 审核跳过：课堂为空 trace=%s path=%s node=%s", trace_id, state.get("path_id"), stat…
        return {"review_passed": False, "review_score": 0, "review_feedback": "课堂结构为空", "review_issues": [], "retry_c…

    segments_by_id = {
        str(segment.get("id")): segment
        for segment in raw_lesson.get("segments", [])
        if isinstance(segment, dict) and str(segment.get("id")) in _SEGMENT_IDS
    }
    missing_ids = [scene_id for scene_id in _SEGMENT_IDS if scene_id not in segments_by_id]
    if missing_ids:
        issues = [{"segment_id": scene_id, "category": "structure", "message": "缺少课堂模块"} for scene_id in missing_ids]
        retry_count = state.get("retry_count", 0)
        can_retry = retry_count < state.get("max_retries", _MAX_REVIEW_RETRIES)
        logger.warning(
            "[ClassroomReviewer] 结构审核失败 trace=%s path=%s node=%s missing=%s retry=%s can_retry=%s",
            trace_id,
            state.get("path_id"),
            state.get("node_id"),
```

文件：backend/src/ai_core/classroom_graph.py

```
            ",".join(missing_ids),
            retry_count,
            can_retry,
        )
        return {
            "review_passed": False,
            "review_score": 0,
            "review_feedback": "课堂模块不完整",
            "review_issues": issues if can_retry else [],
            "retry_count": retry_count + 1 if can_retry else retry_count,
        }

    structural_issues: list[dict[str, Any]] = []
    for scene_id, segment in segments_by_id.items():
        script = str(segment.get("teacher_speech") or segment.get("script") or "").strip()
        board_items = segment.get("board_items") if isinstance(segment.get("board_items"), list) else []
        points = segment.get("points") if isinstance(segment.get("points"), list) else []
        question = segment.get("question") if isinstance(segment.get("question"), dict) else {}
        if not str(segment.get("title") or "").strip() or len(script) < 30 or len(board_items) < 2 or len(points) < 2…
            structural_issues.append({"segment_id": scene_id, "category": "structure", "message": "讲稿、要点或互动问题不完整"})
    concept_issue = _concept_definition_contract_issue(
        segments_by_id.get("concept", {}),
        state.get("knowledge_tags", []),
    )
    if concept_issue:
        structural_issues.append(concept_issue)
    if structural_issues:
        retry_count = state.get("retry_count", 0)
        can_retry = retry_count < state.get("max_retries", _MAX_REVIEW_RETRIES)
        logger.warning(
            "[ClassroomReviewer] 展示契约失败 trace=%s path=%s node=%s scenes=%s retry=%s can_retry=%s",
            trace_id,
            state.get("path_id"),
            state.get("node_id"),
            ",".join(str(item.get("segment_id")) for item in structural_issues),
            retry_count,
            can_retry,
        )
        return {
            "review_passed": False,
            "review_score": 0,
            "review_feedback": "课堂结构不完整",
            "review_issues": structural_issues if can_retry else [],
            "retry_count": retry_count + 1 if can_retry else retry_count,
        }

    user_id_int = int(state.get("user_id", 0))
    llm_priority = state.get("llm_priority", "high")
    logger.info("[ClassroomReviewer] 并行审核开始 trace=%s path=%s node=%s scenes=%s", trace_id, state.get("path_id"), stat…

```

文件：backend/src/ai_core/classroom_graph.py

```
558     async def review_scene(scene_id: str) -> tuple[str, bool, float, str, list[dict]]:
559         review_started_at = time.perf_counter()
560         prompt_text = fill_prompt(
561             load_prompt("classroom/reviewer_segment"),
562             subject=state.get("subject", "未知"),
563             topic=state.get("topic", ""),
564             summary=state.get("summary", ""),
565             knowledge_tags=json.dumps(state.get("knowledge_tags", []), ensure_ascii=False),
566             resources_json=json.dumps(state.get("resources", []), ensure_ascii=False)[:1800],
567             evidence_terms=json.dumps(
568                 _content_evidence_terms(
569                     state.get("topic", ""),
570                     state.get("summary", ""),
571                     *(state.get("knowledge_tags", []) or []),
572                 ),
573                 ensure_ascii=False,
574             ),
575             scene_id=scene_id,
576             scene_responsibility=_SCENE_RESPONSIBILITIES[scene_id],
577             segment_json=json.dumps(segments_by_id[scene_id], ensure_ascii=False)[:2600],
578         )
579         try:
580             logger.info("[ClassroomReviewer] 单幕请求模型 trace=%s path=%s node=%s scene=%s", trace_id, state.get("path_id"…
581             response = await llm.ainvoke(prompt_text, priority=llm_priority, user_id=user_id_int, pool="classroom")
582             result = parse_llm_json(response.content)
583             if not isinstance(result, dict):
584                 result = {}
585             raw_passed, score, feedback, issues = _parse_review_result(result)
586             normalized_issues = [
587                 {"segment_id": scene_id, "category": str(issue.get("category") or "quality"), "message": str(issue.ge…
588                 for issue in issues if isinstance(issue, dict)
589             ]
590             passed = raw_passed and score >= _REVIEW_PASS_SCORE
591             if not passed and not normalized_issues:
592                 normalized_issues = [{"segment_id": scene_id, "category": "quality", "message": feedback or "当前模块未达到审…
593             logger.info(
594                 "[ClassroomReviewer] 单幕审核完成 trace=%s path=%s node=%s scene=%s passed=%s score=%.1f issues=%s elapsed=…
595                 trace_id,
596                 state.get("path_id"),
597                 state.get("node_id"),
598                 scene_id,
599                 passed,
600                 score,
601                 len(normalized_issues),
602                 time.perf_counter() - review_started_at,
603             )
604             return scene_id, passed, score, feedback, normalized_issues
605         except Exception:
606             logger.exception("[ClassroomReviewer] 分幕审核失败，保留当前幕 trace=%s path=%s node=%s scene=%s elapsed=%.2fs", trac…
607             return scene_id, True, 0.0, "", []
```

文件：backend/src/ai_core/classroom_graph.py

```
608
609     scene_reviews = await asyncio.gather(*(review_scene(scene_id) for scene_id in _SEGMENT_IDS))
610     passed = all(item[1] for item in scene_reviews)
611     score = min((item[2] for item in scene_reviews), default=0.0)
612     feedback = "\n".join(item[3] for item in scene_reviews if item[3])[:_REVIEW_FEEDBACK_MAX_LEN]
613     issues = [issue for item in scene_reviews for issue in item[4]]
614     retry_count = state.get("retry_count", 0)
615     actionable_issues = [
616         issue for issue in issues
617         if isinstance(issue, dict) and str(issue.get("segment_id")) in _SEGMENT_IDS
618     ]
619     can_retry = not passed and retry_count < state.get("max_retries", _MAX_REVIEW_RETRIES) and bool(actionable_issues)
620     next_retry_count = retry_count + 1 if can_retry else retry_count
621     logger.info(
622         "[ClassroomReviewer] 审核完成 trace=%s path=%s node=%s passed=%s score=%.1f issues=%s targeted=%s retry=%s elapse…
623         trace_id,
624         state.get("path_id"),
625         state.get("node_id"),
626         passed,
627         score,
628         len(issues),
629         ",".join(str(item.get("segment_id")) for item in actionable_issues) or "none",
630         next_retry_count,
631         time.perf_counter() - started_at,
632     )
633     return {
634         # 没有精确模块的问题时不允许全课重写，保留当前完整课堂。
635         "review_passed": passed or not can_retry,
636         "review_score": score,
637         "review_feedback": feedback[:_REVIEW_FEEDBACK_MAX_LEN] if not passed else "",
638         "review_issues": actionable_issues if can_retry else [],
639         "retry_count": next_retry_count,
640     }
641
642
643 def _parse_review_result(result: dict) -> tuple[bool, float, str, list[dict]]:
644     """容错解析审核结果：passed 兼容 bool/str，score 取 float，issues 非 list 则置 []。"""
645     passed = result.get("passed", False)
646     if isinstance(passed, str):
647         passed = passed.lower() in ("true", "yes", "1", "是", "pass", "通过")
648     try:
649         score = float(result.get("score", 0))
650     except (TypeError, ValueError):
651         score = 0.0
652     feedback = str(result.get("feedback") or "")
653     issues = result.get("issues")
654     if not isinstance(issues, list):
655         issues = []
656     return bool(passed), score, feedback, issues
657
```

文件：backend/src/ai_core/classroom_graph.py

```
658
659 def _concept_definition_contract_issue(segment: dict, knowledge_tags: list[Any]) -> dict | None:
660     """拦截把例子、结论或数值当作概念定义的核心讲解。"""
661     if not isinstance(segment, dict):
662         return {"segment_id": "concept", "category": "concept_definition", "message": "核心概念模块缺失"}
663
664     speech = str(segment.get("teacher_speech") or segment.get("script") or "").strip()
665     opening = speech[:190]
666     definition_pattern = re.compile(
667         r"(?:^|[。；;，,])[^。；;，,]{0,30}?(?:是(?:一种|指|用来|用于)|指的是|用于|表示的是)"
668     )
669     definition_count = len(definition_pattern.findall(opening))
670     if definition_count < 1:
671         return {
672             "segment_id": "concept",
673             "category": "concept_definition",
674             "message": "核心概念必须先用“X 是一种…/X 指…/X 用于…”明确下定义，不能直接从例子、数值或结果开始。",
675         }
676
677     targets: list[str] = []
678     for tag in knowledge_tags if isinstance(knowledge_tags, list) else []:
679         term = str(tag or "").strip()
680         if 2 <= len(term) <= 32 and term not in targets:
681             targets.append(term)
682     required_definitions = 2 if len(targets) >= 2 else 1
683     if definition_count < required_definitions:
684         return {
685             "segment_id": "concept",
686             "category": "concept_definition",
687             "message": "核心概念包含多个术语时，需分别定义至少两个术语，再讲它们的关系和例子。",
688         }
689     return None
690
691
692 # ═══════════════════════════════════════════════════════════
693 #  Router / Graph
694 # ═══════════════════════════════════════════════════════════
695
696 def should_continue(state: ClassroomState) -> str:
697     if state.get("review_passed"):
698         logger.info("[ClassroomGraph] 审核通过或无需全课重写，结束 trace=%s path=%s node=%s", state.get("trace_id", "-"), state.get…
699         return "end"
700     issues = state.get("review_issues") or []
701     if issues:
702         logger.info("[ClassroomGraph] 定向优化模块 trace=%s path=%s node=%s scenes=%s", state.get("trace_id", "-"), state.g…
703         return "writer"
704     logger.warning("[ClassroomGraph] 审核未通过但无可修复模块，结束 trace=%s path=%s node=%s", state.get("trace_id", "-"), state.get…
705     return "end"
706
707
```

文件：backend/src/ai_core/classroom_graph.py

```
def build_classroom_graph():
    workflow = StateGraph(ClassroomState)
    workflow.add_node("writer", writer_node)
    workflow.add_node("reviewer", reviewer_node)
    workflow.add_edge(START, "writer")
    workflow.add_edge("writer", "reviewer")
    workflow.add_conditional_edges(
        "reviewer",
        should_continue,
        {"writer": "writer", "end": END},
    )
    return workflow.compile()


classroom_graph = build_classroom_graph()
```

文件：backend/src/ai_core/llm_config.py

```
"""LLM 配置 + 优先级限流 — 前台请求优先于后台预生成，每用户每池独立并发"""
import asyncio
import hashlib
import json as _json
import logging
import threading
from pathlib import Path
from langchain_openai import ChatOpenAI
from langchain_core.messages import AIMessage
from dotenv import load_dotenv
import os

load_dotenv(Path(__file__).parent.parent.parent / ".env")

logger = logging.getLogger(__name__)

# 通用生成使用独立的 AI_API_KEY，未配置时回退到项目原有的 api_key。
# MiMo 密钥只供视觉模型使用，避免不同供应商之间串用密钥。
api_key = (
    os.getenv("AI_API_KEY")
    or os.getenv("api_key")
)

def _build_chat_model(**kwargs) -> ChatOpenAI | None:
    key = kwargs.get("api_key")
    if not key:
        logger.warning("LLM API key is not configured; AI calls will fail until api_key is set.")
        return None
    return ChatOpenAI(**kwargs)


_raw_llm = _build_chat_model(
    model=os.getenv("AI_MODEL", "deepseek-v4-flash"),
    api_key=api_key,
    base_url=os.getenv("AI_BASE_URL", "https://api.deepseek.com"),
```

文件：backend/src/ai_core/llm_config.py

```
    temperature=0.3,
    streaming=True,
    request_timeout=120, # 单次请求超时 120 秒，避免断连后无限等待
)

# 多模态 LLM（MiMo，用于视觉审查 PPT 截图等）
_vision_llm = _build_chat_model(
    model=os.getenv("VISION_MODEL", "mimo-v2.5"),
    api_key=os.getenv("VISION_API_KEY", api_key),
    base_url=os.getenv("VISION_BASE_URL", "https://api.xiaomimimo.com/v1"),
    temperature=0.3,
    streaming=False,
)

# 每用户每池并发上限（峰值按 5 用户同时使用设计，总并发 ≤500）
_PER_USER = {
    "ppt": 20,       # 主力：单个 PPT 请求最多约 19 条章节线，5 用户约 100 路
    "document": 20,  # 文档生成（实际受 ThreadPool 限制，20 已够）
    "path": 8,       # 学习路径，低频
    "leader": 3,     # 大纲规划，单次调用，不需多路
    "reviewer": 50,  # 审核，5 用户 ×50=250，对齐 _review_sem
    "classroom": 5,  # 互动课堂：1 规划 + 4 幕并行 + 1 审核
    "transition": 1, # 课堂等待页：单用户串行，避免过渡摘要放大并发
    "thread": 10,    # 其他同步任务
}
_DEFAULT_PER_USER = 5

_LLM_LOW_PRI_LIMIT = 10    # 有前台请求时，低优先级最多占 10 路

# 每用户每池并发池（异步）
_user_pool_async: dict[tuple[int, str], asyncio.Semaphore] = {}
_user_pool_async_lock = asyncio.Lock()

# 每用户每池并发池（同步 / 线程）
_user_pool_sync: dict[tuple[int, str], threading.BoundedSemaphore] = {}
_user_pool_sync_lock = threading.Lock()

# 优先级限流（全局）：有前台请求时，低优先级全局限流
_async_low = asyncio.Semaphore(_LLM_LOW_PRI_LIMIT)
_async_high_active = 0
_async_high_lock = asyncio.Lock()

_sync_low = threading.BoundedSemaphore(_LLM_LOW_PRI_LIMIT)
_sync_high_active = 0
_sync_high_lock = threading.Lock()


def _get_user_sync_sem(user_id: int, pool: str = "default") -> threading.BoundedSemaphore | None:
    if not user_id:
        return None
```

文件：backend/src/ai_core/llm_config.py

```
    key = (user_id, pool)
    if key in _user_pool_sync:
        return _user_pool_sync[key]
    with _user_pool_sync_lock:
        if key not in _user_pool_sync:
            limit = _PER_USER.get(pool, _DEFAULT_PER_USER)
            _user_pool_sync[key] = threading.BoundedSemaphore(limit)
        return _user_pool_sync[key]


class _PriorityLLM:
    """LLM 代理：每用户每池独立并发 + 全局优先级限流 + 可选的 Redis 响应缓存。"""

    def __init__(self, raw_llm=None, cache_ttl: int = 0):
        self._raw = raw_llm or _raw_llm
        self._cache_ttl = cache_ttl  # 0=不缓存；>0 缓存秒数（如 3600=1h）

    def __getattr__(self, name):
        if self._raw is None:
            raise RuntimeError("AI model is not configured. Set api_key in backend/.env.")
        return getattr(self._raw, name)

    @staticmethod
    def _prompt_to_key(prompt) -> str | None:
        """将 prompt 转为 SHA256 缓存 key（仅纯文本 prompt 可缓存）"""
        try:
            raw = str(prompt)
            if len(raw) < 10:
                return None
            return hashlib.sha256(raw.encode("utf-8")).hexdigest()
        except Exception:
            return None

    async def ainvoke(self, prompt, priority: str = "high", user_id: int = 0, pool: str = "default"):
        # 计算缓存 key（仅在 cache_ttl > 0 时）
        _cache_key_str = self._prompt_to_key(prompt) if self._cache_ttl else None

        # 缓存快速通道：命中直接返回，不走 semaphore
        if _cache_key_str:
            try:
                from backend.src.utils.redis_client import cache_get as _cg, _cache_key as _ck
                _cached = await _cg(_ck("llm", _cache_key_str))
                if _cached is not None and isinstance(_cached, dict) and "content" in _cached:
                    return AIMessage(
                        content=_cached["content"],
                        response_metadata=_cached.get("response_metadata", {}),
                    )
            except Exception:
                logger.debug("Suppressed exception at backend/src/ai_core/llm_config.py:116", exc_info=True)

```

文件：backend/src/ai_core/llm_config.py

```
136         user_sem = None
137         if user_id:
138             key = (user_id, pool)
139             if key not in _user_pool_async:
140                 async with _user_pool_async_lock:
141                     if key not in _user_pool_async:
142                         limit = _PER_USER.get(pool, _DEFAULT_PER_USER)
143                         _user_pool_async[key] = asyncio.Semaphore(limit)
144             user_sem = _user_pool_async[key]
145 
146         async def _call():
147             if self._raw is None:
148                 raise RuntimeError("AI model is not configured. Set api_key in backend/.env.")
149             if user_sem:
150                 async with user_sem:
151                     resp = await self._raw.ainvoke(prompt)
152             else:
153                 resp = await self._raw.ainvoke(prompt)
154             # 异步回填缓存（仅非流式结果）
155             if _cache_key_str and resp and resp.content and len(str(resp.content).strip()) > 5:
156                 try:
157                     from backend.src.utils.redis_client import cache_set as _cs, _cache_key as _ck2
158                     _meta = getattr(resp, "response_metadata", {}) or {}
159                     await _cs(_ck2("llm", _cache_key_str), {
160                         "content": resp.content,
161                         "response_metadata": {k: str(v) for k, v in _meta.items() if isinstance(v, (str, int, float))…
162                     }, self._cache_ttl)
163                 except Exception:
164                     logger.debug("Suppressed exception at backend/src/ai_core/llm_config.py:144", exc_info=True)
165             return resp
166 
167         global _async_high_active
168         if priority == "high":
169             async with _async_high_lock:
170                 _async_high_active += 1
171             try:
172                 return await _call()
173             finally:
174                 async with _async_high_lock:
175                     _async_high_active -= 1
176         else:
177             async with _async_high_lock:
178                 throttled = _async_high_active > 0
179             if throttled:
180                 async with _async_low:
181                     return await _call()
182             else:
183                 return await _call()
184 
185     def invoke(self, prompt, priority: str = "high", user_id: int = 0, pool: str = "default"):
```

文件：backend/src/ai_core/llm_config.py

```
        user_sem = _get_user_sync_sem(user_id, pool)

        def _call():
            if self._raw is None:
                raise RuntimeError("AI model is not configured. Set api_key in backend/.env.")
            if user_sem:
                with user_sem:
                    return self._raw.invoke(prompt)
            else:
                return self._raw.invoke(prompt)

        global _sync_high_active
        if priority == "high":
            with _sync_high_lock:
                _sync_high_active += 1
            try:
                return _call()
            finally:
                with _sync_high_lock:
                    _sync_high_active -= 1
        else:
            with _sync_high_lock:
                throttled = _sync_high_active > 0
            if throttled:
                with _sync_low:
                    return _call()
            else:
                return _call()


# 环境变量 LLM_CACHE_TTL 控制 LLM 缓存秒数（默认 0=关闭，设置如 300=5分钟）
_DEFAULT_CACHE_TTL = int(os.getenv("LLM_CACHE_TTL", "0"))

llm = _PriorityLLM(cache_ttl=_DEFAULT_CACHE_TTL)
llm_vision = _PriorityLLM(_vision_llm, cache_ttl=_DEFAULT_CACHE_TTL)
```

文件：backend/src/ai_core/path_graph.py

```
"""
LangGraph 多智能体编排 — 个性化学习路径生成
LeaderAgent(规划大纲) → ExecutorAgent(并行分组生成节点) → ReviewerAgent(审核)
"""
import asyncio
import json
import logging
import time
from typing import TypedDict, NotRequired

from langgraph.graph import StateGraph, START, END

from backend.src.ai_core.llm_config import llm
from backend.src.utils.prompt_loader import load_prompt, fill_prompt
from backend.src.utils.json_parser import parse_llm_json
```

文件：backend/src/ai_core/path_graph.py

```
16
17  logger = logging.getLogger(__name__)
18
19  # 每组最多生成的节点数（并行分组）
20  _GROUP_SIZE = 4
21  _MAX_GROUP_CONCURRENCY = 2
22  _GROUP_RETRY_ATTEMPTS = 2
23
24
25  _COGNITIVE_LEVELS = ["记忆", "理解", "应用", "分析", "评价", "创造"]
26
27
28  def _safe_int(value, default: int = 0) -> int:
29      try:
30          return int(value)
31      except (TypeError, ValueError):
32          return default
33
34
35  def _normalize_topic_outline(raw, subject: str, node_count: int = 0) -> list[dict]:
36      if not isinstance(raw, list):
37          return []
38
39      normalized: list[dict] = []
40      seen: set[str] = set()
41      for index, item in enumerate(raw, 1):
42          if not isinstance(item, dict):
43              continue
44          topic = str(item.get("topic") or item.get("title") or "").strip()
45          if not topic or topic in seen:
46              continue
47          seen.add(topic)
48          level = str(item.get("cognitive_level") or _COGNITIVE_LEVELS[min(index - 1, len(_COGNITIVE_LEVELS) - 1)])
49          normalized.append({
50              "topic": topic,
51              "module": str(item.get("module") or item.get("category") or subject).strip(),
52              "cognitive_level": level,
53              "learning_goal": str(item.get("learning_goal") or f"理解并掌握{topic}").strip(),
54              "prerequisite_topics": item.get("prerequisite_topics") if isinstance(item.get("prerequisite_topics"), lis…
55              "key_points": item.get("key_points") if isinstance(item.get("key_points"), list) else [topic],
56              "micro_example": str(item.get("micro_example") or f"完成一个关于{topic}的小练习").strip(),
57          })
58          if node_count and len(normalized) >= node_count:
59              break
60      return normalized
61
62
63  def _fallback_topic_outline(subject: str, node_count: int = 0) -> list[dict]:
64      target = max(8, min(18, _safe_int(node_count, 12) or 12))
65      templates = [
```

文件：frontend/src/utils/markdown.js

```
  let paragraph = []
  let listItems = []
  let listType = ''
  let codeLines = []
  let inCodeBlock = false
  let codeLanguage = ''

  const flushParagraph = () => {
    if (!paragraph.length) return
    html.push(`<p>${paragraph.map(renderInlineMarkdown).join('<br>')}</p>`)
    paragraph = []
  }

  const flushList = () => {
    if (!listItems.length) return
    const tag = listType === 'ol' ? 'ol' : 'ul'
    html.push(`<${tag}>${listItems.map(item => `<li>${renderInlineMarkdown(item)}</li>`).join('')}</${tag}>`)
    listItems = []
    listType = ''
  }

  const flushCode = () => {
    const languageLabel = codeLanguage ? `<span>${escapeHtml(codeLanguage)}</span>` : ''
    html.push(`<div class="md-code-block">${languageLabel}<pre><code>${escapeHtml(codeLines.join('\n'))}</code></pre>…
    codeLines = []
    codeLanguage = ''
  }

  for (let index = 0; index < lines.length; index += 1) {
    const rawLine = lines[index]
    const line = rawLine.trim()

    if (line.startsWith('```')) {
      if (inCodeBlock) {
        flushCode()
        inCodeBlock = false
      } else {
        flushParagraph()
        flushList()
        inCodeBlock = true
        codeLanguage = line.replace(/^```/, '').trim()
      }
      continue
    }

    if (inCodeBlock) {
      codeLines.push(rawLine)
      continue
    }

```

文件：frontend/src/utils/markdown.js

```
    if (!line) {
      flushParagraph()
      flushList()
      continue
    }

    if (line.includes('|') && lines[index + 1] && isTableSeparator(lines[index + 1])) {
      flushParagraph()
      flushList()
      const tableLines = [rawLine, lines[index + 1]]
      index += 2
      while (index < lines.length && lines[index].includes('|') && lines[index].trim()) {
        tableLines.push(lines[index])
        index += 1
      }
      index -= 1
      html.push(renderTable(tableLines))
      continue
    }

    const headingMatch = line.match(/^(#{1,4})\s+(.+)$/)
    if (headingMatch) {
      flushParagraph()
      flushList()
      const level = Math.min(headingMatch[1].length + 2, 6)
      html.push(`<h${level}>${renderInlineMarkdown(headingMatch[2])}</h${level}>`)
      continue
    }

    const blockquoteMatch = line.match(/^>\s?(.+)$/)
    if (blockquoteMatch) {
      flushParagraph()
      flushList()
      html.push(`<blockquote>${renderInlineMarkdown(blockquoteMatch[1])}</blockquote>`)
      continue
    }

    const orderedMatch = line.match(/^\d+\.\s+(.+)$/)
    const unorderedMatch = line.match(/^[-*]\s+(.+)$/)

    if (orderedMatch || unorderedMatch) {
      flushParagraph()
      const currentType = orderedMatch ? 'ol' : 'ul'

      if (listType && listType !== currentType) {
        flushList()
      }

      listType = currentType
      listItems.push((orderedMatch?.[1] || unorderedMatch?.[1] || '').trim())
```

文件：frontend/src/utils/markdown.js

```
      continue
    }

    flushList()
    paragraph.push(rawLine)
  }

  if (inCodeBlock) {
    flushCode()
  }
  flushParagraph()
  flushList()

  return renderMath(html.join(''))
}
```

文件：frontend/src/utils/pptistAdapter.js

```
import { mergeSpeakerNotes, splitSpeakerNotes, stripSpeakerNotes } from './pptSpeakerNotes'

export const PPTIST_PAYLOAD_KEY = 'zhiban:pptist:payload'

const WIDTH = 1000
const HEIGHT = 562.5

const PALETTES = {
  default: ['#163f8f', '#5f8fc3', '#c9dce9', '#ffffff'],
  minimal: ['#111216', '#3b3f4a', '#6b6f7a', '#ffffff'],
  dark: ['#7aa2f7', '#bb9af7', '#7dcfff', '#1a1b26'],
  warm: ['#d94860', '#e36a2d', '#f2a341', '#fff7ef'],
  green: ['#11695f', '#28b487', '#a7f3d0', '#f6fffb']
}

const escapeHtml = value => String(value || '')
  .replace(/&/g, '&amp;')
  .replace(/</g, '&lt;')
  .replace(/>/g, '&gt;')
  .replace(/"/g, '&quot;')
  .replace(/'/g, '&#39;')

const cleanText = value => String(value || '')
  .replace(/<!--[\s\S]*?-->/g, ' ')
  .replace(/<\/?[^>\n]+>/g, ' ')
  .replace(/^#{1,6}\s+/gm, '')
  .replace(/^[\-*+・]\s+/gm, '')
  .replace(/\*\*(.*?)\*\*/g, '$1')
  .replace(/\n{3,}/g, '\n\n')
  .trim()

const splitBlocks = value => cleanText(value)
  .split(/\r?\n/)
  .map(line => line.trim())
  .filter(Boolean)
```

文件：frontend/src/utils/pptistAdapter.js

```
    .slice(0, 8)

const estimateLines = (value, charsPerLine = 33) => Math.max(1, Math.ceil(String(value || '').length / charsPerLine))

const paginateBlocks = (blocks, {
  maxLines = 12,
  maxItems = 5,
  charsPerLine = 33
} = {}) => {
  const pages = []
  let current = []
  let lineCount = 0

  for (const block of blocks) {
    const text = cleanText(block)
    if (!text) continue

    const blockLines = estimateLines(text, charsPerLine)
    const shouldStartNewPage = current.length && (
      lineCount + blockLines > maxLines ||
      current.length >= maxItems
    )

    if (shouldStartNewPage) {
      pages.push(current)
      current = []
      lineCount = 0
    }

    current.push(text)
    lineCount += blockLines
  }

  if (current.length) pages.push(current)
  return pages.length ? pages : [[]]
}

const htmlParagraph = (value, { size = 24, color = '#333333', bold = false, align = 'left' } = {}) => {
  const tagOpen = bold ? '<strong>' : ''
  const tagClose = bold ? '</strong>' : ''
  return `<p style="text-align:${align};">${tagOpen}<span style="font-size:${size}px;color:${color};">${escapeHtml(va…
}

const htmlList = (items, { size = 22, color = '#333333' } = {}) => {
  if (!items.length) return htmlParagraph('', { size, color })
  return `<ul style="font-size:${size}px;color:${color};">${items.map(item => `<li><p>${escapeHtml(item)}</p></li>`).…
}

const textElement = (id, content, left, top, width, height, options = {}) => ({
  type: 'text',
```

文件：frontend/src/utils/pptistAdapter.js

```
    id,
    left,
    top,
    width,
    height,
    rotate: 0,
    content,
    defaultFontName: options.font || '',
    defaultColor: options.color || '#333333',
    lineHeight: options.lineHeight || 1.25,
    fixedHeight: Boolean(options.fixedHeight),
    vAlign: options.vAlign || 'top',
    ...(options.fill ? { fill: options.fill } : {}),
    ...(options.textType ? { textType: options.textType } : {}),
    ...(options.inset ? { inset: options.inset } : {})
  })

  const rectElement = (id, left, top, width, height, fill, options = {}) => ({
    type: 'shape',
    id,
    left,
    top,
    width,
    height,
    viewBox: [200, 200],
    path: 'M 0 0 L 200 0 L 200 200 L 0 200 Z',
    fill,
    fixedRatio: false,
    rotate: 0,
    ...(options.opacity ? { opacity: options.opacity } : {}),
    ...(options.radius ? { radius: options.radius } : {})
  })

  const paletteForTheme = themeId => {
    const id = String(themeId || '').toLowerCase()
    if (id.includes('dark') || id.includes('night') || id.includes('mocha') || id.includes('terminal')) return PALETTES…
    if (id.includes('warm') || id.includes('sunset') || id.includes('retro')) return PALETTES.warm
    if (id.includes('green') || id.includes('science')) return PALETTES.green
    if (id.includes('minimal') || id.includes('white')) return PALETTES.minimal
    return PALETTES.default
  }

  const normalizeSlide = slide => {
    const title = cleanText(slide?.title || slide?.heading || 'Untitled')
    const bodyParts = splitSpeakerNotes(slide?.text || slide?.content || slide?.body || '')
    const blockNotes = (Array.isArray(slide?.blocks) ? slide.blocks : [])
      .map(block => splitSpeakerNotes(block?.text || block?.content || block).notes)
      .filter(Boolean)
    const blocks = Array.isArray(slide?.blocks) && slide.blocks.length
      ? slide.blocks.map(block => cleanText(stripSpeakerNotes(block?.text || block?.content || block))).filter(Boolean)
```

文件：frontend/src/utils/pptistAdapter.js

```
    : splitBlocks(bodyParts.body)
  const notes = cleanText(mergeSpeakerNotes(slide?.notes, slide?.speaker_notes, bodyParts.notes, ...blockNotes))
  return { title, blocks, notes }
}

const buildCoverSlide = (slide, index, palette) => {
  const [primary, secondary, accent, paper] = palette
  const id = `zhiban_slide_${index + 1}`
  return {
    id,
    background: { type: 'solid', color: paper },
    remark: slide.notes || '',
    elements: [
      rectElement(`${id}_band`, 0, 0, 1000, 562.5, primary, { opacity: 0.08 }),
      rectElement(`${id}_left`, 0, 0, 18, 562.5, primary),
      rectElement(`${id}_accent`, 72, 365, 250, 8, accent),
      textElement(`${id}_title`, htmlParagraph(slide.title, { size: 46, color: primary, bold: true }), 72, 145, 820, ···
        color: primary,
        textType: 'title',
        fixedHeight: true,
        vAlign: 'middle'
      }),
      textElement(`${id}_sub`, htmlParagraph(slide.blocks.slice(0, 2).join('  '), { size: 20, color: secondary }), 76···
        color: secondary,
        textType: 'subtitle'
      })
    ]
  }
}

const buildContentSlide = (slide, index, palette) => {
  const [primary, secondary, accent, paper] = palette
  const id = `zhiban_slide_${index + 1}`
  const blocks = slide.blocks.length ? slide.blocks : ['No content']
  const estimatedLines = blocks.reduce((sum, block) => sum + estimateLines(block), 0)
  const dense = estimatedLines > 10 || blocks.length > 4
  const bodyFontSize = estimatedLines > 13 ? 17 : estimatedLines > 10 ? 18 : blocks.length > 5 ? 19 : 22
  const bodyWidth = dense ? 820 : 600
  return {
    id,
    background: { type: 'solid', color: paper },
    remark: slide.notes || '',
    elements: [
      rectElement(`${id}_top`, 0, 0, 1000, 72, primary, { opacity: 0.92 }),
      rectElement(`${id}_accent`, 64, 104, 68, 6, accent),
      textElement(`${id}_title`, htmlParagraph(slide.title, { size: 30, color: '#ffffff', bold: true }), 58, 14, 850,···
        color: '#ffffff',
        textType: 'title',
        fixedHeight: true,
        vAlign: 'middle'
```

文件：frontend/src/utils/pptistAdapter.js

```
      }),
      textElement(`${id}_body`, htmlList(blocks, { size: bodyFontSize, color: '#1f2937' }), 74, 124, bodyWidth, 390, {
        color: '#1f2937',
        textType: 'content',
        lineHeight: dense ? 1.18 : 1.3,
        fixedHeight: true
      }),
      ...(!dense ? [
        rectElement(`${id}_visual`, 720, 135, 208, 248, secondary, { opacity: 0.14 }),
        rectElement(`${id}_visual_bar`, 720, 135, 208, 14, secondary),
        textElement(`${id}_visual_text`, htmlParagraph(slide.title, { size: 20, color: primary, bold: true, align: 'c…
          color: primary,
          fixedHeight: true,
          vAlign: 'middle'
        })
      ] : [])
    ]
  }
}

const expandSlidesForPptist = slides => {
  const expanded = []

  slides.forEach((slide, slideIndex) => {
    if (slideIndex === 0) {
      expanded.push(slide)
      return
    }

    const pages = paginateBlocks(slide.blocks, {
      maxLines: 12,
      maxItems: 5,
      charsPerLine: 34
    })

    pages.forEach((blocks, pageIndex) => {
      expanded.push({
        ...slide,
        title: pageIndex ? `${slide.title} (cont.)` : slide.title,
        blocks
      })
    })
  })

  return expanded
}

export const toPptistPayload = ({ title = 'Zhiban PPT', slides = [], themeId = '' } = {}) => {
  const palette = paletteForTheme(themeId)
  const normalizedSlides = (Array.isArray(slides) ? slides : [])
```

文件：frontend/src/utils/pptistAdapter.js

```
    .map(normalizeSlide)
    .filter(slide => slide.title || slide.blocks.length)
  const pptistSlides = expandSlidesForPptist(normalizedSlides)

  return {
    title,
    viewportSize: WIDTH,
    viewportRatio: HEIGHT / WIDTH,
    theme: {
      themeColors: palette.slice(0, 3).concat(['#ffc000', '#4472c4', '#70ad47']),
      fontColor: '#333333',
      fontName: '',
      backgroundColor: palette[3],
      shadow: { h: 3, v: 3, blur: 2, color: '#808080' },
      outline: { width: 2, color: palette[0], style: 'solid' }
    },
    slides: pptistSlides.map((slide, index) => (
      index === 0
        ? buildCoverSlide(slide, index, palette)
        : buildContentSlide(slide, index, palette)
    ))
  }
}

export const openPptistEditor = payload => {
  const key = `${PPTIST_PAYLOAD_KEY}:${Date.now()}`
  localStorage.setItem(key, JSON.stringify(payload))
  window.open(`/pptist/index.html?source=${encodeURIComponent(key)}`, '_blank', 'noopener,noreferrer')
}
```

文件：frontend/src/utils/pptSpeakerNotes.js

```
const NOTE_LABEL_RE = /^(?:\u8bb2\u7a3f|\u5907\u6ce8|\u6f14\u8bb2\u7a3f|speaker\s*notes?|speaker_notes?|notes?)\s*[:\…

const unwrapLine = line => String(line || '')
  .replace(/^\s*(?:[-*+\u2022]\s*)?/, '')
  .replace(/^\s*>\s?/, '')
  .trim()

const matchNoteLine = line => unwrapLine(line).match(NOTE_LABEL_RE)

export const splitSpeakerNotes = value => {
  const lines = String(value || '').split(/\r?\n/)
  const body = []
  const notes = []
  let readingQuotedNote = false

  for (const rawLine of lines) {
    const matched = matchNoteLine(rawLine)
    if (matched) {
      const noteText = matched[1]?.trim()
      if (noteText) notes.push(noteText)
      readingQuotedNote = /^\s*>/.test(rawLine)
```

文件：frontend/src/utils/pptSpeakerNotes.js

```
      continue
    }

    if (readingQuotedNote && /^\s*>/.test(rawLine)) {
      const noteText = unwrapLine(rawLine)
      if (noteText) notes.push(noteText)
      continue
    }

    readingQuotedNote = false
    body.push(rawLine)
  }

  return {
    body: body.join('\n').trim(),
    notes: notes.join('\n').trim()
  }
}

export const stripSpeakerNotes = value => splitSpeakerNotes(value).body

export const mergeSpeakerNotes = (...values) => values
  .map(value => String(value || '').trim())
  .filter(Boolean)
  .join('\n')
```

文件：frontend/src/utils/quizBank.d.ts

```
export type QuizOption = {
  key: string
  text: string
}

export type QuizQuestion = {
  id: string
  type: 'choice' | 'short'
  stem: string
  options: QuizOption[]
  answer: string
  explanation: string
}

export type QuizSet = {
  id: string
  sourceId: string
  title: string
  content: string
  fileType: string
  filename: string
  questionCount: number
  questions: QuizQuestion[]
  createdAt: string
}
```

文件：frontend/src/utils/quizBank.d.ts

```
26
27  export const QUIZ_BANK_KEY: string
28
29  export function parseQuizQuestions(content: string): QuizQuestion[]
30
31  export function looksLikeQuizContent(content: string): boolean
32
33  export function upsertQuizSet(payload: {
34    id?: string
35    sourceId?: string | number
36    title?: string
37    content?: string
38    fileType?: string
39    filename?: string
40    questions?: QuizQuestion[]
41    createdAt?: string
42  }): QuizSet | null
43
44  export function readQuizBank(): QuizSet[]
45
46  export function getQuizSet(quizId: string): QuizSet | null
```

文件：frontend/src/utils/quizBank.js

```
1   const SESSION_KEY = 'zhiban_quiz_sessions'
2   // v3: 题面必须与后端会话记录一致，旧版本地缓存可能保存了重排前的选项。
3   export const QUIZ_SCHEMA_VERSION = 3
4
5   const uid = prefix => `${prefix}-${Date.now()}-${Math.random().toString(16).slice(2)}`
6
7   const readSessions = () => {
8     try {
9       const data = JSON.parse(localStorage.getItem(SESSION_KEY) || '[]')
10      return Array.isArray(data) ? data : []
11    } catch {
12      return []
13    }
14  }
15
16  const writeSessions = sessions => {
17    localStorage.setItem(SESSION_KEY, JSON.stringify(sessions))
18  }
19
20  const stripFence = value =>
21    String(value || '')
22      .trim()
23      .replace(/^```(?:json|markdown|md)?\s*/i, '')
24      .replace(/```$/i, '')
25      .trim()
26
27  const normalizeAnswer = value => {
28    if (typeof value === 'boolean') return value ? 'A' : 'B'
29    if (Array.isArray(value)) return value.map(k => String(k || '').trim().toUpperCase()).filter(Boolean).sort().join('…
```

文件：frontend/src/utils/quizBank.js

```
  const text = String(value ?? '').trim()
  // 处理 JSON 序列化后的布尔和字母
  if (/^(?:true|false)$/i.test(text)) return text.toUpperCase() === 'TRUE' ? 'A' : 'B'
  try {
    const parsed = JSON.parse(text)
    if (typeof parsed === 'boolean') return parsed ? 'A' : 'B'
    if (Array.isArray(parsed)) return parsed.map(k => String(k || '').trim().toUpperCase()).filter(Boolean).sort().jo…
  } catch {}
  return text
    .replace(/^(答案|正确答案|参考答案)[:：]?\s*/i, '')
    .replace(/^[（(]?\s*([A-F])\s*[）).、]?\s*$/i, '$1')
    .toUpperCase()
}

const normalizeQuestion = (item, index) => {
  const rawOptions = item.options || item.choices || item.option || []
  const options = Array.isArray(rawOptions)
    ? rawOptions.map((option, optionIndex) => {
      const text = typeof option === 'string'
        ? option
        : String(option.text || option.content || option.value || '')
      const match = text.match(/^\s*([A-F])[\).、]\s*(.*)$/i)

      return {
        key: String(option.key || option.label || match?.[1] || String.fromCharCode(65 + optionIndex)).toUpperCase(),
        text: match?.[2] || text
      }
    })
    : []

  // 只用字符串类型的值作为题干，跳过 object（如 record 包装格式 {question: {...}}）
  const stem = (() => {
    for (const key of ['stem', 'question', 'content', 'title', 'question_text']) {
      const val = item[key]
      if (typeof val === 'string') return val
    }
    return `第 ${index + 1} 题`
  })()

  const questionType = String(item.question_type || item.type || '').toLowerCase()
  const isMulti =
    item.multi ||
    item.is_multi ||
    item.multiple ||
    ['multiple', 'multi', 'checkbox', 'multi_choice'].includes(questionType)

  const rawAnswer = item.answer ?? item.correctAnswer ?? item.correct_answer ?? item.correct ?? ''
  const answerStr = String(rawAnswer || '').trim()
  const hasMultiple = /[,，、]/.test(answerStr) || /^[A-F]{2,}$/i.test(answerStr)

```

文件：frontend/src/utils/quizBank.js

```
    const multi = isMulti || hasMultiple

    return {
      question_id: item.question_id || item.questionId || item.id || null,
      id: item.question_id || item.questionId || item.id || uid(`question-${index + 1}`),
      type: questionType || (multi ? 'multiple' : (options.length ? 'choice' : 'short')),
      question_type: questionType || (multi ? 'multi_choice' : (options.length ? 'single_choice' : 'fill_blank')),
      stem: String(stem).trim(),
      options,
      answer: multi
        ? answerStr.toUpperCase().replace(/[^A-F,]/g, '').split(/[,，、]+/).filter(Boolean).sort().join(',')
        : normalizeAnswer(answerStr),
      multi,
      explanation: String(item.explanation || item.analysis || item.reason || '').trim()
    }
  }

  const parseJsonQuestions = content => {
    const text = stripFence(content)
    const candidates = [text]
    const arrayMatch = text.match(/\[[\s\S]*\]/)
    const objectMatch = text.match(/\{[\s\S]*\}/)

    if (arrayMatch) candidates.push(arrayMatch[0])
    if (objectMatch) candidates.push(objectMatch[0])

    for (const candidate of candidates) {
      try {
        const parsed = JSON.parse(candidate)
        const list = Array.isArray(parsed) ? parsed : parsed.questions || parsed.items || parsed.data || []
        if (Array.isArray(list) && list.length) {
          // 解包 record 包装格式 {record_id, question: {...}} → {...}
          const unwrapped = list.map(item =>
            item.question && typeof item.question === 'object' ? item.question : item
          )
          return unwrapped.map(normalizeQuestion).filter(item => item.stem)
        }
      } catch {
        // Try next candidate.
      }
    }

    return []
  }

  const parseMarkdownQuestions = content => {
    const text = stripFence(content)
    const blocks = text
      .split(/\n(?=\s*(?:#{1,4}\s*)?(?:第\s*)?\d+\s*[.、）)]|\n-{3,}\n)/)
      .map(block => block.trim())
```

文件：frontend/src/utils/quizBank.js

```
    .filter(Boolean)

  const sourceBlocks = blocks.length > 1
    ? blocks
    : text.split(/\n{2,}/).map(block => block.trim()).filter(Boolean)

  return sourceBlocks.map((block, index) => {
    const lines = block.split(/\r?\n/).map(line => line.trim()).filter(Boolean)
    const stemLines = []
    const options = []
    let answer = ''
    let explanation = ''

    lines.forEach(line => {
      const optionMatch = line.match(/^(?:[-*]\s*)?([A-D])[\).、]\s*(.+)$/i)
      const answerMatch = line.match(/^(?:[-*]\s*)?(?:答案|正确答案|参考答案|answer)[:：]\s*(.+)$/i)
      const explanationMatch = line.match(/^(?:[-*]\s*)?(?:解析|解释|说明|analysis)[:：]\s*(.+)$/i)

      if (optionMatch) {
        options.push({ key: optionMatch[1].toUpperCase(), text: optionMatch[2].trim() })
        return
      }

      if (answerMatch) {
        answer = answerMatch[1].trim()
        return
      }

      if (explanationMatch) {
        explanation = explanationMatch[1].trim()
        return
      }

      stemLines.push(line.replace(/^#{1,4}\s*/, '').replace(/^(?:第\s*)?\d+\s*[.、）)]\s*/, ''))
    })

    return normalizeQuestion({ stem: stemLines.join('\n'), options, answer, explanation }, index)
  }).filter(item => item.stem && (item.options.length || item.answer))
}

export const parseQuizQuestions = content => {
  const jsonQuestions = parseJsonQuestions(content)
  if (jsonQuestions.length) return jsonQuestions
  return parseMarkdownQuestions(content)
}

export const looksLikeQuizContent = content => {
  const text = String(content || '')
  if (!text.trim()) return false
  if (parseQuizQuestions(text).length > 0) return true
```

文件：frontend/src/utils/quizBank.js

```
  return /question_type|single_choice|multi_choice|true_false|正确答案|参考答案|答案[:：]|解析[:：]/i.test(text)
}

export const QUIZ_BANK_KEY = SESSION_KEY
export const QUIZ_ATTEMPTS_KEY = 'zhiban_quiz_attempts'

export const upsertQuizSet = payload => {
  const questions = payload.questions || parseQuizQuestions(payload.content)
  if (!questions.length) return null

  const sessionId =
    payload.id ||
    (payload.sourceId ? `quiz-resource-${payload.sourceId}` : uid('quiz'))

  const session = {
    schemaVersion: QUIZ_SCHEMA_VERSION,
    id: sessionId,
    sourceId: payload.sourceId || '',
    sessionId: payload.sessionId || payload.session_id || '',
    title: payload.title || 'AI 生成题目',
    content: payload.content || '',
    fileType: payload.fileType || 'exercise',
    filename: payload.filename || '',
    questionCount: questions.length,
    questions,
    attempts: payload.attempts || [],
    bestScore: payload.bestScore ?? null,
    lastScore: payload.lastScore ?? null,
    lastAttemptAt: payload.lastAttemptAt || '',
    createdAt: payload.createdAt || new Date().toISOString()
  }

  const sessions = readSessions()
  const next = [session, ...sessions.filter(item => item.id !== sessionId && item.sourceId !== session.sourceId)]
  writeSessions(next)
  window.dispatchEvent(new CustomEvent('zhiban-quiz-bank-updated', { detail: session }))
  return session
}

export const readQuizBank = () => readSessions()

export const getQuizSet = quizId => {
  const session = readSessions().find(item => item.id === quizId) || null
  // 清理已缓存的老格式垃圾数据（record 包装 → stem 成了 [object Object]）
  if (session?.questions?.length) {
    const clean = session.questions.filter(q => q.stem && q.stem !== '[object Object]')
    if (clean.length !== session.questions.length) {
      const sessions = readSessions()
      if (clean.length === 0) {
        // 全部损坏，移除整个 session，让上游重新生成
```

文件：frontend/src/utils/quizBank.js

```
        const next = sessions.filter(s => s.id !== session.id)
        writeSessions(next)
        return null
      }
      session.questions = clean
      session.questionCount = clean.length
      const next = sessions.map(s => s.id === session.id ? session : s)
      writeSessions(next)
    }
  }
  return session
}

const readAttempts = () => {
  try {
    const data = JSON.parse(localStorage.getItem(QUIZ_ATTEMPTS_KEY) || '[]')
    return Array.isArray(data) ? data : []
  } catch {
    return []
  }
}

const writeAttempts = attempts => {
  localStorage.setItem(QUIZ_ATTEMPTS_KEY, JSON.stringify(attempts))
}

export const readQuizAttempts = quizId => {
  const attempts = readAttempts()
  return quizId ? attempts.filter(item => item.quizId === quizId) : attempts
}

export const recordQuizAttempt = payload => {
  const quizId = String(payload.quizId || '')
  if (!quizId) return null

  const attempt = {
    id: uid('attempt'),
    quizId,
    sessionId: payload.sessionId || '',
    title: payload.title || '',
    score: Number(payload.score || 0),
    total: Number(payload.total || 0),
    percent: Number(payload.percent || 0),
    answers: payload.answers || {},
    results: payload.results || {},
    createdAt: new Date().toISOString()
  }

  const attempts = [attempt, ...readAttempts()]
  writeAttempts(attempts)
```

文件：frontend/src/utils/quizBank.js

```
  const sessions = readSessions()
  const next = sessions.map(session => {
    if (session.id !== quizId) return session
    const sessionAttempts = [attempt, ...(session.attempts || [])]
    const bestScore = Math.max(Number(session.bestScore || 0), attempt.percent)
    return {
      ...session,
      attempts: sessionAttempts,
      bestScore,
      lastScore: attempt.percent,
      lastAttemptAt: attempt.createdAt
    }
  })
  writeSessions(next)
  window.dispatchEvent(new CustomEvent('zhiban-quiz-bank-updated', { detail: getQuizSet(quizId) }))
  return attempt
}
```

文件：frontend/src/utils/renderMath.js

```
import katex from 'katex'

const render = (formula, displayMode) => {
  try {
    return katex.renderToString(formula.trim(), {
      displayMode,
      throwOnError: false,
      trust: true,
    })
  } catch {
    return formula
  }
}

/**
 * 修复 LLM 常见公式问题：
 * 1. 裸 \begin{...}...\end{...} 没包 $ → 补 $$...$$
 * 2. $ 放反：A$ → $A$
 */
function fixBrokenLatex(text) {
  // 1. 清理 \begin{env}...\end{env} 周围游离的 $，统一用 $$ 包裹
  //    LLM 常见错误：裸奔、$ 放一边、$...$ 内嵌块级环境
  text = text.replace(
    /\$*\\begin\{([a-zA-Z*]+)\}[\s\S]*?\\end\{\1\}\$*/g,
    (match) => `$$${match.replace(/^\$+|\$+$/g, '')}$$`,
  )
  // 2. $ 放反：数学表达式后紧跟 $ 但前面无 $ 配对 → 补开头的 $
  return text.replace(
    /(^|[^$\\{}^_])([A-Za-z0-9×÷±≤≥≠≈·ΣΠ∫√∞∂∇+\-*=<>]+)\$(?=[\s，。、；：,.;:()（）、—-⊠])/g,
    '$1$$2$$',
  )
}
```

文件：frontend/src/utils/renderMath.js

```
33
34  /** CJK 及中文标点 — 这些字符不应出现在纯数学公式中 */
35  const NON_MATH_RE = /[一-龥㐀-䶵豈-﫿 -⁯⺀-⻿]/
36  const CJK_STRIP_RE = /[一-龥㐀-䶵豈-﫿 -⁯⺀-⻿，。、；：！？ （） 【】 《》 "'"']/g
37
38  /** 标点/空白 — 不需要 karaoke 高亮，也不消耗 timedWords */
39  const PUNCT_RE = /^[\s，。、；：！？ ,.;:!?]+$/
40
41  /**
42   * 将文本渲染为带 karaoke 逐词高亮的 HTML。
43   * 与 renderMath 一样处理 LaTeX，同时将普通文字按 timedWords 状态包入 karaoke span。
44   *
45   * @param {string} text - 待渲染文本
46   * @param {Array<{text: string, isDone: boolean, isCurrent: boolean}>} timedWords - 全 slide 的词状态
47   * @param {number} wordOffset - 当前 item 在 timedWords 中的起始下标
48   * @returns {{ html: string, consumed: number }}
49   */
50  export function renderKaraokeHTML(text, timedWords, wordOffset = 0) {
51    if (!text || typeof text !== 'string') return { html: text, consumed: 0 }
52
53    const fixed = fixBrokenLatex(text)
54    const words = timedWords || []
55    let wi = wordOffset
56    let consumedTotal = 0
57    let result = ''
58
59    const parts = fixed.split(/(\$\$[\s\S]*?\$\$|\\\[[\s\S]*?\\\]|\$[^\s\$][^\$]*?\$|\\\([\s\S]*?\\\))/g)
60
61    for (const part of parts) {
62      if (!part) continue
63
64      // LaTeX 公式 — 直接渲染，不消耗 timedWords 位置
65      if (/^\$\$|^\\\[|^\$|^\\\(/.test(part)) {
66        if (part.startsWith('$$') || part.startsWith('\\[')) {
67          const f = part.replace(/^\$\$|\$\$/g, '').replace(/^\\\[|\\\]$/g, '')
68          result += render(f, true)
69        } else {
70          const f = part.replace(/^\$|\$/g, '').replace(/^\\\(|\\\)$/g, '')
71          if (NON_MATH_RE.test(f)) {
72            const stripped = f.replace(CJK_STRIP_RE, '').trim()
73            if (stripped && stripped.length > 1) {
74              try { result += render(stripped, false) } catch { result += `$${f}$` }
75            } else {
76              result += `$${f}$`
77            }
78          } else {
79            result += render(f, false)
80          }
81        }
82        continue
```

文件：frontend/src/utils/renderMath.js

```
    }

    // 逐字拆分 — timedWords 已由上层插值为字级，每个 Unicode 字符对应一条
    const chars = [...part]
    for (const ch of chars) {
      if (PUNCT_RE.test(ch)) {
        result += ch
        continue
      }
      const tw = wi < words.length ? words[wi] : null
      // timedWords 不足时，宁可保持未高亮，也不要把后面的字误判成已完成
      // 这样可以避免“段尾文字在一开始就直接亮起来”的错位现象。
      const isDone = tw ? tw.isDone : false
      const isCurrent = tw ? tw.isCurrent : false
      const cls = `karaoke-word${isDone ? ' is-done' : ''}${isCurrent ? ' is-current' : ''}`
      result += `<span class="${cls}">${ch}</span>`
      wi++
      consumedTotal++
    }
  }

  return { html: result, consumed: consumedTotal }
}

/**
 * 将文本中的 LaTeX 公式渲染为 HTML。
 * 支持：$$...$$ / \[...\] （块级），$...$ / \(...\) （行内）
 */
export function renderMath(text) {
  if (!text || typeof text !== 'string') return text

  const fixed = fixBrokenLatex(text)

  // 按顺序处理：块级优先，避免 $ 干扰 $$
  return fixed
    .replace(/\$\$([\s\S]*?)\$\$/g, (_, f) => render(f, true))
    .replace(/\\\[([\s\S]*?)\\\]/g, (_, f) => render(f, true))
    .replace(/\$([^\s\$][^\$]*)\$/g, (_, f) => {
      if (NON_MATH_RE.test(f)) {
        // 尝试剥离中文后渲染数学部分
        const stripped = f.replace(CJK_STRIP_RE, '').trim()
        if (stripped && stripped.length > 1) {
          try {
            return render(stripped, false)
          } catch {
            return `$${f}$`
          }
        }
        return `$${f}$` // 纯中文，不渲染
      }
```

文件：frontend/src/utils/renderMath.js

```
      return render(f, false)
    })
    .replace(/\\\(([\s\S]*?)\\\)/g, (_, f) => render(f, false))
}
```

文件：frontend/src/utils/resourceCover.js

```
import { resolveApiUrl } from '../api/apis'

const palettes = [
  ['#2f5d9f', '#58aeea', '#e9f7fb'],
  ['#1e8f8c', '#55c9a7', '#e9fbf6'],
  ['#8f5bbf', '#d9a7ff', '#f8f0ff'],
  ['#c96f22', '#f2b705', '#fff7df'],
  ['#d94d7b', '#ef8fb0', '#fff0f5'],
  ['#566bd8', '#8eb1ff', '#eff4ff']
]

const documentMotifs = [
  { icon: 'notes', side: '#f2b705' },
  { icon: 'chart', side: '#55c9a7' },
  { icon: 'stack', side: '#d9a7ff' },
  { icon: 'quote', side: '#ef8fb0' }
]

const typeLabels = {
  document: '文档',
  reference: '文档',
  ppt: 'PPT',
  powerpoint: 'PPT',
  presentation: 'PPT',
  mindmap: '导图',
  exercise: '题库',
  quiz: '题库',
  image: '图片',
  video: '视频'
}

const escapeXml = value => String(value || '')
  .replace(/&/g, '&amp;')
  .replace(/</g, '&lt;')
  .replace(/>/g, '&gt;')
  .replace(/"/g, '&quot;')

const hashText = value => {
  const text = String(value || '')
  let hash = 0
  for (let index = 0; index < text.length; index += 1) {
    hash = ((hash << 5) - hash + text.charCodeAt(index)) | 0
  }
  return Math.abs(hash)
}

```

文件：frontend/src/utils/resourceCover.js

```
47  const compactTitle = value => {
48    const title = String(value || '学习资源').replace(/\s+/g, ' ').trim()
49    return title.length > 18 ? `${title.slice(0, 18)}...` : title
50  }
51
52  const splitTitleLines = value => {
53    const title = String(value || '学习资源').replace(/\s+/g, ' ').trim()
54    if (title.length <= 10) return [title]
55    return [title.slice(0, 10), title.slice(10, 20)]
56  }
57
58  const resourceKind = resource => {
59    const text = String(`${resource?.type || ''} ${resource?.category || ''} ${resource?.categoryLabel || ''} ${resourc…
60    if (/image|图片/.test(text)) return 'image'
61    if (/video|mp4|视频/.test(text)) return 'video'
62    if (/mind|xmind|导图/.test(text)) return 'mindmap'
63    if (/ppt|powerpoint|presentation|slide/.test(text)) return 'ppt'
64    if (/exercise|quiz|question|exam|题|练习/.test(text)) return 'exercise'
65    return 'document'
66  }
67
68  const toDataSvg = svg => `data:image/svg+xml;charset=UTF-8,${encodeURIComponent(svg.trim())}`
69
70  const renderPptCover = ({ primary, accent, rawTitle }) => {
71    const [lineOne, lineTwo = ''] = splitTitleLines(rawTitle).map(escapeXml)
72    return toDataSvg(`
73      <svg xmlns="http://www.w3.org/2000/svg" width="520" height="280" viewBox="0 0 520 280">
74        <defs>
75          <linearGradient id="pptBg" x1="0" x2="1" y1="0" y2="1">
76            <stop offset="0%" stop-color="${primary}"/>
77            <stop offset="56%" stop-color="${accent}"/>
78            <stop offset="100%" stop-color="#ffffff"/>
79          </linearGradient>
80          <filter id="shadow" x="-20%" y="-20%" width="140%" height="140%">
81            <feDropShadow dx="0" dy="10" stdDeviation="10" flood-color="#163f8f" flood-opacity="0.18"/>
82          </filter>
83        </defs>
84        <rect width="520" height="280" rx="20" fill="url(#pptBg)"/>
85        <circle cx="452" cy="42" r="92" fill="#ffffff" opacity="0.14"/>
86        <circle cx="64" cy="246" r="116" fill="#ffffff" opacity="0.12"/>
87        <rect x="54" y="42" width="354" height="196" rx="12" fill="#ffffff" filter="url(#shadow)"/>
88        <rect x="54" y="42" width="354" height="48" rx="12" fill="${primary}"/>
89        <rect x="54" y="78" width="354" height="12" fill="${primary}"/>
90        <text x="78" y="72" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="20" font-weight="900" fil…
91        <text x="78" y="136" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="28" font-weight="900" fi…
92        <text x="78" y="172" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="24" font-weight="800" fi…
93        <rect x="78" y="196" width="142" height="9" rx="4.5" fill="${accent}" opacity="0.62"/>
94        <rect x="78" y="216" width="238" height="9" rx="4.5" fill="${primary}" opacity="0.18"/>
95        <g transform="translate(424 76)">
96          <rect x="0" y="0" width="58" height="40" rx="6" fill="#ffffff" opacity="0.9"/>
```

文件：frontend/src/utils/resourceCover.js

```
      <rect x="9" y="9" width="28" height="5" rx="2.5" fill="${primary}" opacity="0.65"/>
      <rect x="9" y="20" width="40" height="5" rx="2.5" fill="${accent}" opacity="0.55"/>
      <rect x="-18" y="58" width="58" height="40" rx="6" fill="#ffffff" opacity="0.74"/>
      <rect x="-9" y="67" width="32" height="5" rx="2.5" fill="${primary}" opacity="0.5"/>
      <rect x="-9" y="78" width="42" height="5" rx="2.5" fill="${accent}" opacity="0.48"/>
    </g>
  </svg>
  `)
}

const renderDocumentCover = ({ primary, accent, paper, rawTitle, resource }) => {
  const [lineOne, lineTwo = ''] = splitTitleLines(rawTitle).map(escapeXml)
  const motif = documentMotifs[hashText(rawTitle || resource?.doc_id) % documentMotifs.length]
  const motifSvg = {
    notes: `
      <rect x="0" y="0" width="86" height="12" rx="6" fill="${primary}" opacity="0.24"/>
      <rect x="0" y="28" width="118" height="12" rx="6" fill="${accent}" opacity="0.3"/>
      <rect x="0" y="56" width="72" height="12" rx="6" fill="${primary}" opacity="0.18"/>
    `,
    chart: `
      <rect x="0" y="38" width="18" height="42" rx="6" fill="${primary}" opacity="0.74"/>
      <rect x="30" y="14" width="18" height="66" rx="6" fill="${accent}" opacity="0.74"/>
      <rect x="60" y="28" width="18" height="52" rx="6" fill="${motif.side}" opacity="0.82"/>
      <path d="M0 16L30 2l30 16 34-12" fill="none" stroke="${primary}" stroke-width="6" stroke-linecap="round" opacit…
    `,
    stack: `
      <rect x="0" y="12" width="118" height="76" rx="12" fill="${primary}" opacity="0.16"/>
      <rect x="14" y="0" width="118" height="76" rx="12" fill="${accent}" opacity="0.24"/>
      <rect x="28" y="-12" width="118" height="76" rx="12" fill="#ffffff" opacity="0.76"/>
      <rect x="44" y="12" width="70" height="8" rx="4" fill="${primary}" opacity="0.26"/>
      <rect x="44" y="32" width="92" height="8" rx="4" fill="${accent}" opacity="0.32"/>
    `,
    quote: `
      <text x="0" y="70" font-family="Georgia, serif" font-size="92" font-weight="900" fill="${primary}" opacity="0.1…
      <rect x="48" y="18" width="88" height="10" rx="5" fill="${accent}" opacity="0.34"/>
      <rect x="48" y="42" width="118" height="10" rx="5" fill="${primary}" opacity="0.2"/>
      <rect x="48" y="66" width="68" height="10" rx="5" fill="${motif.side}" opacity="0.42"/>
    `
  }[motif.icon]

  return toDataSvg(`
    <svg xmlns="http://www.w3.org/2000/svg" width="520" height="280" viewBox="0 0 520 280">
      <defs>
        <linearGradient id="docBg" x1="0" x2="1" y1="0" y2="1">
          <stop offset="0%" stop-color="${paper}"/>
          <stop offset="46%" stop-color="#ffffff"/>
          <stop offset="100%" stop-color="#eef6ff"/>
        </linearGradient>
        <linearGradient id="docBand" x1="0" x2="1">
          <stop offset="0%" stop-color="${primary}"/>
```

文件：frontend/src/utils/resourceCover.js

```
        <stop offset="52%" stop-color="${accent}"/>
        <stop offset="100%" stop-color="${motif.side}"/>
      </linearGradient>
      <filter id="docShadow" x="-20%" y="-20%" width="140%" height="140%">
        <feDropShadow dx="0" dy="12" stdDeviation="12" flood-color="#17345f" flood-opacity="0.16"/>
      </filter>
    </defs>
    <rect width="520" height="280" rx="24" fill="url(#docBg)"/>
    <path d="M0 0h520v80H0z" fill="url(#docBand)"/>
    <path d="M342 0h178v280H342z" fill="${primary}" opacity="0.06"/>
    <path d="M390 0c62 30 96 78 102 144 5 60-17 101-66 124h94V0z" fill="${accent}" opacity="0.12"/>
    <rect x="42" y="38" width="304" height="204" rx="18" fill="#ffffff" filter="url(#docShadow)"/>
    <rect x="42" y="38" width="304" height="44" rx="18" fill="#f7fbff"/>
    <path d="M42 64h304" stroke="${primary}" stroke-opacity="0.1" stroke-width="2"/>
    <rect x="66" y="58" width="76" height="10" rx="5" fill="${primary}" opacity="0.18"/>
    <rect x="264" y="56" width="42" height="14" rx="7" fill="${motif.side}" opacity="0.28"/>
    <text x="66" y="124" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="26" font-weight="900" fi…
    <text x="66" y="158" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="22" font-weight="800" fi…
    <rect x="66" y="184" width="218" height="8" rx="4" fill="${primary}" opacity="0.16"/>
    <rect x="66" y="206" width="176" height="8" rx="4" fill="${accent}" opacity="0.24"/>
    <rect x="66" y="224" width="238" height="8" rx="4" fill="${primary}" opacity="0.1"/>
    <g transform="translate(374 126)">
      <rect x="-18" y="-24" width="118" height="104" rx="20" fill="#ffffff" opacity="0.78"/>
      ${motifSvg}
    </g>
    <rect x="380" y="34" width="84" height="28" rx="14" fill="#ffffff" opacity="0.92"/>
    <text x="422" y="53" text-anchor="middle" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="15"…
    <path d="M42 38h18v204H42z" fill="${motif.side}" opacity="0.82"/>
  </svg>
  `)
}

const renderGenericCover = ({ primary, accent, paper, kind, rawTitle }) => {
  const title = escapeXml(compactTitle(rawTitle))
  const label = escapeXml(typeLabels[kind] || '资源')

  return toDataSvg(`
    <svg xmlns="http://www.w3.org/2000/svg" width="520" height="280" viewBox="0 0 520 280">
      <defs>
        <linearGradient id="bg" x1="0" x2="1" y1="0" y2="1">
          <stop offset="0%" stop-color="${paper}"/>
          <stop offset="100%" stop-color="#ffffff"/>
        </linearGradient>
        <linearGradient id="band" x1="0" x2="1">
          <stop offset="0%" stop-color="${primary}"/>
          <stop offset="100%" stop-color="${accent}"/>
        </linearGradient>
      </defs>
      <rect width="520" height="280" rx="28" fill="url(#bg)"/>
      <path d="M0 0h520v86H0z" fill="url(#band)" opacity="0.95"/>
```

文件：frontend/src/utils/resourceCover.js

```
      <circle cx="440" cy="56" r="74" fill="#fff" opacity="0.16"/>
      <circle cx="488" cy="26" r="36" fill="#fff" opacity="0.14"/>
      <rect x="34" y="36" width="92" height="34" rx="17" fill="#fff" opacity="0.9"/>
      <text x="80" y="59" text-anchor="middle" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="18" …
      <text x="34" y="150" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="32" font-weight="900" fi…
      <rect x="34" y="188" width="260" height="10" rx="5" fill="${primary}" opacity="0.18"/>
      <rect x="34" y="212" width="360" height="10" rx="5" fill="${primary}" opacity="0.12"/>
      <rect x="34" y="236" width="190" height="10" rx="5" fill="${primary}" opacity="0.12"/>
      <path d="M398 162c34 0 62 28 62 62h-62z" fill="${accent}" opacity="0.55"/>
      <path d="M398 224h62c0 34-28 62-62 62z" fill="${primary}" opacity="0.82" transform="translate(0 -62)"/>
    </svg>
  `)
}

const renderIllustratedResourceCover = ({ primary, accent, paper, kind, rawTitle, resource }) => {
  const [lineOne, lineTwo = ''] = splitTitleLines(rawTitle).map(escapeXml)
  const seed = hashText(rawTitle || resource?.doc_id || resource?.sourceId || kind)
  const warm = ['#f2b705', '#ef8fb0', '#55c9a7', '#8eb1ff'][seed % 4]
  const cool = ['#e9fbf6', '#eff4ff', '#fff7df', '#f8f0ff'][seed % 4]
  const accentTwo = ['#1e8f8c', '#566bd8', '#c96f22', '#8f5bbf'][seed % 4]
  const motif = {
    ppt: `
      <g transform="translate(318 66)">
        <rect x="24" y="20" width="132" height="92" rx="18" fill="#ffffff" opacity="0.7"/>
        <rect x="0" y="0" width="132" height="92" rx="18" fill="#ffffff" filter="url(#shadow)"/>
        <path d="M18 60c26-34 44-18 58-36 20 20 34 40 38 54H18z" fill="${accent}" opacity="0.3"/>
        <circle cx="94" cy="28" r="12" fill="${warm}" opacity="0.86"/>
        <rect x="18" y="18" width="52" height="8" rx="4" fill="${primary}" opacity="0.22"/>
        <rect x="18" y="36" width="76" height="7" rx="3.5" fill="${accentTwo}" opacity="0.24"/>
      </g>
    `,
    mindmap: `
      <g transform="translate(326 58)" fill="none" stroke-linecap="round">
        <path d="M72 76L26 34M72 76l76-36M72 76l-18 76M72 76l88 76" stroke="${primary}" stroke-width="8" opacity="0.2…
        <circle cx="72" cy="76" r="28" fill="#ffffff" stroke="${primary}" stroke-width="0" filter="url(#shadow)"/>
        <circle cx="26" cy="34" r="18" fill="${accent}" opacity="0.8"/>
        <circle cx="148" cy="40" r="22" fill="${warm}" opacity="0.86"/>
        <circle cx="54" cy="152" r="20" fill="${accentTwo}" opacity="0.78"/>
        <circle cx="160" cy="152" r="18" fill="#ffffff" opacity="0.82"/>
      </g>
    `,
    exercise: `
      <g transform="translate(330 62)">
        <rect x="10" y="0" width="132" height="160" rx="24" fill="#ffffff" filter="url(#shadow)"/>
        <path d="M34 46l16 16 34-38" fill="none" stroke="${accentTwo}" stroke-width="10" stroke-linecap="round" strok…
        <rect x="34" y="86" width="84" height="10" rx="5" fill="${primary}" opacity="0.18"/>
        <rect x="34" y="112" width="64" height="10" rx="5" fill="${warm}" opacity="0.42"/>
        <circle cx="122" cy="34" r="32" fill="${accent}" opacity="0.16"/>
      </g>
    `,
```

文件：frontend/src/utils/resourceCover.js

```
    document: `
      <g transform="translate(322 54)">
        <rect x="22" y="20" width="118" height="154" rx="22" fill="${primary}" opacity="0.08"/>
        <rect x="0" y="0" width="124" height="160" rx="22" fill="#ffffff" filter="url(#shadow)"/>
        <path d="M92 0v42h32" fill="${cool}"/>
        <path d="M92 0v42h32" fill="none" stroke="${primary}" stroke-opacity="0.1" stroke-width="2"/>
        <rect x="24" y="48" width="58" height="9" rx="4.5" fill="${primary}" opacity="0.18"/>
        <rect x="24" y="74" width="80" height="9" rx="4.5" fill="${accent}" opacity="0.26"/>
        <rect x="24" y="100" width="66" height="9" rx="4.5" fill="${warm}" opacity="0.35"/>
      </g>
    `,
    video: `
      <g transform="translate(330 64)">
        <rect x="0" y="0" width="132" height="96" rx="16" fill="#ffffff" filter="url(#shadow)"/>
        <polygon points="52,28 52,68 92,48" fill="${warm}" opacity="0.86"/>
        <rect x="0" y="106" width="132" height="8" rx="4" fill="${primary}" opacity="0.18"/>
        <rect x="0" y="124" width="76" height="8" rx="4" fill="${accent}" opacity="0.32"/>
        <rect x="84" y="124" width="48" height="8" rx="4" fill="${warm}" opacity="0.22"/>
      </g>
    `
  }[kind] || `
    <g transform="translate(326 62)">
      <rect x="0" y="0" width="148" height="148" rx="34" fill="#ffffff" filter="url(#shadow)"/>
      <path d="M34 112c18-54 48-84 96-96" fill="none" stroke="${accent}" stroke-width="16" stroke-linecap="round" opa…
      <circle cx="48" cy="48" r="20" fill="${warm}" opacity="0.86"/>
      <circle cx="104" cy="92" r="28" fill="${primary}" opacity="0.16"/>
    </g>
  `

  return toDataSvg(`
    <svg xmlns="http://www.w3.org/2000/svg" width="520" height="280" viewBox="0 0 520 280">
      <defs>
        <linearGradient id="learningBg" x1="0" x2="1" y1="0" y2="1">
          <stop offset="0%" stop-color="${paper}"/>
          <stop offset="52%" stop-color="#ffffff"/>
          <stop offset="100%" stop-color="${cool}"/>
        </linearGradient>
        <linearGradient id="ribbon" x1="0" x2="1">
          <stop offset="0%" stop-color="${primary}"/>
          <stop offset="100%" stop-color="${accent}"/>
        </linearGradient>
        <filter id="shadow" x="-30%" y="-30%" width="160%" height="160%">
          <feDropShadow dx="0" dy="14" stdDeviation="14" flood-color="#17345f" flood-opacity="0.16"/>
        </filter>
      </defs>
      <rect width="520" height="280" rx="28" fill="url(#learningBg)"/>
      <path d="M0 0h520v76H0z" fill="url(#ribbon)" opacity="0.9"/>
      <circle cx="448" cy="48" r="96" fill="#ffffff" opacity="0.14"/>
      <circle cx="72" cy="242" r="116" fill="${accent}" opacity="0.12"/>
      <path d="M34 212c70-48 134-42 194 18 28 28 58 32 92 12" fill="none" stroke="${primary}" stroke-width="16" strok…
```

文件：frontend/src/utils/resourceCover.js

```
      <rect x="38" y="42" width="236" height="176" rx="24" fill="#ffffff" opacity="0.74" filter="url(#shadow)"/>
      <rect x="62" y="68" width="88" height="10" rx="5" fill="${warm}" opacity="0.64"/>
      <text x="82" y="122" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="24" font-weight="900" fi…
      <text x="82" y="154" font-family="Microsoft YaHei, PingFang SC, sans-serif" font-size="20" font-weight="800" fi…
      <rect x="82" y="182" width="126" height="8" rx="4" fill="${primary}" opacity="0.16"/>
      <rect x="82" y="201" width="166" height="8" rx="4" fill="${accent}" opacity="0.18"/>
      ${motif}
    </svg>
  `)
}

export const getExplicitResourceCoverUrl = resource => {
  const explicitCover =
    resource?.coverUrl ||
    resource?.cover_url ||
    resource?.thumbnailUrl ||
    resource?.thumbnail_url ||
    resource?.thumb_url ||
    ''
  return explicitCover ? resolveApiUrl(explicitCover) : ''
}

const getGeneratedResourceCoverUrl = resource => {
  const kind = resourceKind(resource)
  if (kind === 'image' && resource?.previewUrl) return resolveApiUrl(resource.previewUrl)

  const [primary, accent, paper] = palettes[hashText(resource?.doc_id || resource?.sourceId || resource?.title) % pal…
  const rawTitle = resource?.title || resource?.filename
  const coverContext = { primary, accent, paper, rawTitle, resource, kind }

  return renderIllustratedResourceCover(coverContext)
}

const shouldUseExplicitCover = resource => {
  // 只有图片有真实预览图才用原图做封面，其余全部走生成
  const kind = resourceKind(resource)
  return kind === 'image' && Boolean(resource?.previewUrl)
}

export const getResourceCoverUrl = resource => {
  const explicitCover = getExplicitResourceCoverUrl(resource)
  if (explicitCover && shouldUseExplicitCover(resource)) return explicitCover

  return getGeneratedResourceCoverUrl(resource)
}
```

文件：frontend/src/utils/savedResources.d.ts

```
export const SAVED_GENERATED_RESOURCES_KEY: string;

export interface SavedResourceRef {
  id: string;
  sourceId: string;
```

文件：frontend/src/utils/savedResources.d.ts

```
  kind: string;
  fileType: string;
  category: string;
  quizId: string;
  title: string;
  filename: string;
  content: string;
  coverUrl: string;
  previewUrl: string;
  downloadUrl: string;
  annotations: Array<Record<string, unknown>>;
  visibility: string;
  createdAt: string;
}

export interface SaveGeneratedResourcePayload {
  sourceId?: string | number;
  kind?: string;
  fileType?: string;
  category?: string;
  quizId?: string;
  title?: string;
  filename?: string;
  content?: string;
  coverUrl?: string;
  previewUrl?: string;
  downloadUrl?: string;
  annotations?: Array<Record<string, unknown>>;
  visibility?: string;
}

export function readSavedResourceRefs(visibility?: string): SavedResourceRef[];

export function saveGeneratedResourceRef(payload: SaveGeneratedResourcePayload): SavedResourceRef;

export function hydrateSavedResourceRefs(visibility?: string): Promise<unknown[]>;
```

文件：frontend/src/utils/savedResources.js

```
import { getGeneratedImage, getGeneratedResource, resolveApiUrl } from '../api/apis'
import { getResourceCoverUrl } from './resourceCover'

export const SAVED_GENERATED_RESOURCES_KEY = 'zhiban_saved_generated_resource_refs'

const readJson = key => {
  try {
    const value = JSON.parse(localStorage.getItem(key) || '[]')
    return Array.isArray(value) ? value : []
  } catch {
    return []
  }
}

```

文件：frontend/src/utils/savedResources.js

```
15  const writeJson = (key, value) => {
16    localStorage.setItem(key, JSON.stringify(value))
17  }
18
19  export const readSavedResourceRefs = visibility => {
20    const refs = readJson(SAVED_GENERATED_RESOURCES_KEY)
21    return visibility ? refs.filter(item => item.visibility === visibility) : refs
22  }
23
24  export const saveGeneratedResourceRef = payload => {
25    const sourceId = String(payload.sourceId || payload.quizId || '')
26    if (!sourceId) {
27      throw new Error('生成资源缺少 id，暂时不能保存。')
28    }
29
30    const kind = payload.kind || 'resource'
31    const id = `${kind}-${sourceId}`
32    const record = {
33      id,
34      sourceId,
35      kind,
36      fileType: payload.fileType || 'file',
37      category: payload.category || 'reference',
38      quizId: payload.quizId || '',
39      title: payload.title || '',
40      filename: payload.filename || '',
41      content: payload.content || '',
42      coverUrl: payload.coverUrl || '',
43      previewUrl: payload.previewUrl || '',
44      downloadUrl: payload.downloadUrl || '',
45      annotations: Array.isArray(payload.annotations) ? payload.annotations : [],
46      visibility: payload.visibility || 'private',
47      createdAt: new Date().toISOString()
48    }
49
50    const refs = readSavedResourceRefs()
51    const next = [record, ...refs.filter(item => item.id !== id)]
52    writeJson(SAVED_GENERATED_RESOURCES_KEY, next)
53    window.dispatchEvent(new CustomEvent('zhiban-generated-resource-saved', { detail: record }))
54    return record
55  }
56
57  const unwrap = result => result?.data?.data ?? result?.data ?? result
58
59  const normalizeDetail = (detail, ref) => {
60    const item = unwrap(detail) || {}
61    const title =
62      item.title ||
63      item.filename ||
64      item.file_name ||
```

文件：frontend/src/utils/savedResources.js

```
      item.name ||
      ref.title ||
      ref.filename ||
      `${ref.kind === 'image' ? '生成图片' : '生成资源'}-${ref.sourceId}`
    const fileType = item.file_type || item.fileType || item.resource_type || item.resourceType || ref.fileType || 'fil…
    const content = item.preview || item.content || item.text || item.preview_content || item.previewContent || ref.con…
    const downloadUrl =
      item.download_url ||
      item.downloadUrl ||
      ref.downloadUrl ||
      item.url ||
      (ref.kind === 'image' ? `/image/${ref.sourceId}/download` : ref.quizId ? '' : `/resource/${ref.sourceId}/download…
    const previewUrl =
      item.preview_url ||
      item.previewUrl ||
      ref.previewUrl ||
      item.preview ||
      item.url ||
      item.image_url ||
      item.imageUrl ||
      ''

    const resource = {
      doc_id: `${ref.kind}-${ref.sourceId}`,
      sourceId: ref.sourceId,
      source: 'generated',
      title,
      content,
      type: fileType,
      category: ref.category || (String(fileType).includes('exercise') ? 'exercise' : 'reference'),
      categoryLabel: '',
      visibility: ref.visibility || 'private',
      quizId: ref.quizId || '',
      created_at: item.created_at || item.createdAt || ref.createdAt || '',
      filename: ref.filename || title,
      annotations: Array.isArray(item.annotations) ? item.annotations : (Array.isArray(ref.annotations) ? ref.annotatio…
      coverUrl: resolveApiUrl(item.cover_url || item.coverUrl || item.thumbnail_url || item.thumbnailUrl || ref.coverUr…
      previewUrl: resolveApiUrl(previewUrl),
      downloadUrl: resolveApiUrl(downloadUrl)
    }

    return {
      ...resource,
      coverUrl: getResourceCoverUrl(resource)
    }
  }

  export const hydrateSavedResourceRefs = async visibility => {
    const refs = readSavedResourceRefs(visibility)
    const settled = await Promise.allSettled(
```

文件：frontend/src/utils/savedResources.js

```
    refs.map(async ref => {
      if (ref.quizId) return normalizeDetail({}, ref)
      const detail = ref.kind === 'image'
        ? await getGeneratedImage(ref.sourceId)
        : await getGeneratedResource(ref.sourceId)
      return normalizeDetail(detail, ref)
    })
  )

  return settled.map((item, index) => {
    if (item.status === 'fulfilled') return item.value
    return normalizeDetail({}, refs[index])
  })
}
```

文件：frontend/src/vite-env.d.ts

```
/// <reference types="vite/client" />

declare module "*.vue" {
  import type { DefineComponent } from "vue";

  const component: DefineComponent<Record<string, unknown>, Record<string, unknown>, unknown>;
  export default component;
}

declare module "*.png" {
  const src: string;
  export default src;
}

declare module "*.jpg" {
  const src: string;
  export default src;
}

declare module "*.jpeg" {
  const src: string;
  export default src;
}

declare module "*.webp" {
  const src: string;
  export default src;
}

declare module "*.gif" {
  const src: string;
  export default src;
}

declare module "*.svg" {
  const src: string;
```

文件：frontend/src/vite-env.d.ts

```
37    export default src;
38  }
```